

Computer Vision Lab (AI-4002): Exam Prep Guide, Labs 1–6

Built for Rijaa · Exam: open book, no internet, printed Python/OpenCV cheat sheet allowed · 3 questions, Q1 = MCQs

0. Read this first (how to use this guide)

What this is. A from-zero teaching guide for Labs 1–6. Each lab has: (1) the concepts, (2) every function and parameter that matters, (3) model solutions for the lab tasks, and (4) the traps examiners love. Part 4 at the end has an MCQ bank, a likely-question list, and a printable cheat sheet.

Honesty notes (please read).

- **Lab 2's manual was never uploaded.** Section 2 is built from the Lab 2 *task sheet* plus standard image-processing knowledge (log, gamma, piecewise, histogram). Cross-check the formulas against your teacher's slides if you have them.
- Labs 1 and 6 were scanned images, so I read them via OCR. Tiny typos are possible.
- Where I believe the **manual itself contains an error or loose claim**, I mark it with . That is my own judgement, not a confirmed correction. For MCQs, answer what the *manual* says if the question is clearly manual-based, but know the correct version.
- Anything I'm less than fully sure about is flagged with "I believe". Verify it before you rely on it.
- I can't see your images or run your data, so I never invent experimental numbers (e.g. Otsu thresholds, coin counts). Those you must get by running your own code.

3-pass plan (suggested).

1. *Understand*: read each lab section, run the code snippets once.
2. *Rebuild*: close the guide, write each lab's pipeline from a blank cell.
3. *Trap-hunt*: do the MCQ bank, then reread every  box.

What I'd expect on the exam (my guess from the teacher's announcement, not confirmed). Q1: MCQs on concepts, function parameters, and "what does this code output". Q2/Q3: scenario-style coding tasks like the lab task sheets (load → process → display), probably with short justification.

1. The universal toolkit (used in every single lab)

1.1 The skeleton almost every answer follows

```
import cv2, numpy as np, matplotlib.pyplot as plt

img = cv2.imread('image.jpg')          # BGR, uint8, shape (H, W, 3)
if img is None:                          # imread does NOT raise an error on
    a bad path                           # a bad path
    print('Error: image not found')
else:
    rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)  # Matplotlib expects RGB
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    # ... process ...
    plt.figure(figsize=(12, 5))
    plt.subplot(1, 2, 1); plt.imshow(rgb); plt.title('Original');
plt.axis('off')
    plt.subplot(1, 2, 2); plt.imshow(gray, cmap='gray'); plt.title('Result');
plt.axis('off')
    plt.tight_layout(); plt.show()
```

1.2 Facts that cause most lost marks

Fact	Detail
Channel order	<code>cv2.imread</code> gives BGR , not RGB. Matplotlib wants RGB.
Failed load	<code>imread</code> returns None (no exception). Always check if <code>img</code> is <code>None</code> .
Shape order	<code>img.shape</code> = (height, width, channels). Grayscale: (height, width).
Size arguments	OpenCV functions take (width, height) : <code>resize(dsize=(w,h))</code> , <code>warpAffine(..., (w,h))</code> .
Point arguments	Drawing and geometry use (x, y) : <code>circle(center=(x,y))</code> , <code>putText(org=(x,y))</code> .
NumPy indexing	<code>img[y, x] = img[row, col]</code> . Cropping: <code>img[y1:y2, x1:x2]</code> .
Gray display	Single-channel needs <code>cmap='gray'</code> in <code>plt.imshow</code> , else you get false colours.

Fact	Detail
dtype	Images are <code>uint8</code> (0–255). <code>cv2.add</code> saturates at 255; NumPy <code>a + b</code> wraps (e.g. $200+100 = 44$).
Kernel sizes	<code>GaussianBlur</code> kernel sizes must be odd and positive.
subplot	<code>plt.subplot(rows, cols, index)</code> , index starts at 1 .
Jupyter	<code>cv2.imshow</code> doesn't work inline in notebooks (the Lab 1 manual says so too). Use Matplotlib.

2. Lab 1: Intro to computer vision + basic image operations

2.1 Concepts to know cold

Computer vision = enabling computers to interpret and understand visual information (images/video), extracting meaning and acting on it. It draws on image processing, ML, AI, and statistics.

Terminology (manual list, the most testable ones):

- **Pixel**: smallest unit of an image; **resolution**: dimensions in pixels (e.g. 1920×1080).
- **Grayscale**: one value per pixel, 0 (black) to 255 (white). **RGB**: three values per pixel.
- **Feature**: a distinctive part of an image; **feature descriptor**: compact representation of a feature for matching.
- **Edge detection**: finding boundaries. **Segmentation**: splitting an image into meaningful regions. **Object detection**: find *and locate*. **Classification**: assign a label to the whole image.
- **Image registration**: aligning images of the same scene into one coordinate system. **Homography**: transformation relating two images of the same *planar* surface. **Optical flow**: motion between consecutive frames. **Tracking**: following objects across frames.
- **CNN**: deep model with convolutional layers that learn hierarchical features. **Feature map**: output of a convolutional filter.
- **SIFT** (scale-invariant local features), **SURF** (faster SIFT-like), **HOG** (gradient-orientation descriptor for detection).

Applications: autonomous vehicles, medical imaging, robotics, object detection/recognition, facial recognition, AR/VR.

Libraries (manual's comparison, simplified):

- **OpenCV**: general CV, C++/Python, independent framework, high abstraction.
- **Scikit-Image**: image processing in Python, integrates with NumPy/Matplotlib. **Pillow**: basic image processing.
- **TorchVision**: CV for PyTorch. **TensorFlow/Keras**: Keras is the high-level API (now part of TensorFlow). **PyTorch**: general ML.
- **OpenVINO**: Intel inference optimisation. **Detectron2**: PyTorch object detection. **Caffe**: C++ deep-learning framework focused on CNNs. **Hugging Face**: pretrained NLP and CV models.

Digital image. A 2D function $f(x, y)$ where x, y are spatial coordinates and f is intensity (gray level). "Digital" means x, y and amplitude are all finite and discrete. Stored as an $M \times N$ matrix. Origin (0,0) is the **top-left**.

⚠ Lab 1 labels x = row index (0... $M-1$), y = column index (0... $N-1$). Lab 3 instead says $x \rightarrow$ right, $y \rightarrow$ down (i.e. x = column). Don't mix them: with NumPy always think `img[row, col]`.

Fundamental steps of digital image processing (know the order):

1. Image acquisition $\rightarrow$ 2. Image enhancement $\rightarrow$ 3. Image restoration $\rightarrow$ 4. Colour image processing $\rightarrow$ 5. Wavelets & multiresolution processing $\rightarrow$ 6. Compression $\rightarrow$ 7. Morphological processing $\rightarrow$ 8. Segmentation $\rightarrow$ 9. Representation & description $\rightarrow$ 10. Object recognition. The **knowledge base** interacts with all steps.
- Enhancement = highlight interesting features (brightness, contrast). Restoration = improve appearance (undo degradation). Segmentation = partition into objects (the hardest step). Representation/description = turn segmented raw pixels into descriptors; recognition = assign a label based on descriptors.

Purpose of DIP (5 groups): visualization, sharpening & restoration, image retrieval, measurement of patterns, image recognition.

2.2 The operations, parameter by parameter

Read + display + titled plot

```
plt.figure(figsize=(8, 6)); plt.axis('off')
plt.title('King of Curses - Ryomen Sukuna', fontsize=16, color='darkred',
pad=15)
plt.imshow(rgb); plt.show()
```

`figsize=(w, h)` in inches; `pad` = gap between title and axes.

Grayscale: `cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)` or `cv2.imread(path, cv2.IMREAD_GRAYSCALE)`.

Resize: `cv2.resize(img, None, fx=0.5, fy=0.5)` (scale factors) or `cv2.resize(img, (new_w, new_h))` (absolute size, width first). Default interpolation is `INTER_LINEAR`. Rule of thumb: `INTER_AREA` for shrinking, `INTER_CUBIC` / `INTER_LINEAR` for enlarging.

Gaussian blur: `cv2.GaussianBlur(img, (k, k), sigmaX)`.

- `(k,k)` = kernel size, odd. Larger `k` → stronger blur. `sigmaX=0` → OpenCV derives sigma from the kernel size.
- The manual uses (31, 31) for a heavy blur; the Lab 1 task asks for 25×25.

Crop (pure NumPy slicing): `img[y1:y2, x1:x2]`. Left half: `img[:, :w//2]`. Right half: `img[:, w//2:]`.

Text: `cv2.putText(img, text, org, fontFace, fontScale, color, thickness)`.

- `org` = **bottom-left** corner of the text (x, y). `fontScale` multiplies the base font size. It draws **in place**.
- Colour tuple follows the array's channel order. On an **RGB** array `(255,0,0)` is red; on a **BGR** array red is `(0,0,255)`.
- ⚠ The manual's description says scale 1.5 but the code uses 5. Code wins when output is asked.

Pandas analysis:

```
flat = img.reshape(-1, 3) # (H*W, 3)
df = pd.DataFrame(flat, columns=['B', 'G', 'R']) # imread order is B, G, R!
print(df.describe()) # count, mean, std, min, 25%,
                    50%, 75%, max
```

The Lab 1 task asks for columns `['Red', 'Green', 'Blue']`: convert to RGB first (`rgb.reshape(-1,3)`) so the labels are truthful.

Binary threshold: `ret, out = cv2.threshold(gray, thresh, maxval, type)`

- `THRESH_BINARY`: `out = maxval` if `pixel > thresh` else 0 (strictly greater).
- ⚠ The manual uses `maxval=200`, so "white" is 200, not 255. Returns `(ret, image)`; `ret` is the threshold used.
- Other types: `THRESH_BINARY_INV`, `THRESH_TRUNC`, `THRESH_TOZERO`, `THRESH_TOZERO_INV`, plus `THRESH_OTSU` (added to a type; auto-picks thresh).

Rotation:

```
h, w = img.shape[:2]
M = cv2.getRotationMatrix2D((w/2, h/2), 45, 1) # (centre x,y), angle°,
scale
rot = cv2.warpAffine(img, M, (w, h)) # output size (w, h)
```

- Positive angle = **counter-clockwise** (top-left origin). M is 2×3 .
- Same canvas size $\Rightarrow$ corners get cut off, empty areas are filled black.
- Internally (OpenCV docs, I'm confident): with $\alpha = \text{scale} \cdot \cos\theta$, $\beta = \text{scale} \cdot \sin\theta$, $M = [[\alpha, \beta, (1-\alpha) \cdot cx - \beta \cdot cy], [-\beta, \alpha, \beta \cdot cx + (1-\alpha) \cdot cy]]$.

Blending:

- `cv2.add(a, b)` = per-pixel sum with **saturation** at 255 (result looks brighter). Both images must have the same size and type, so `cv2.resize` one first.
- `cv2.addWeighted(a,  $\alpha$ , b,  $\beta$ ,  $\gamma$ )` = $\alpha \cdot a + \beta \cdot b + \gamma$: the proper *semi-transparent* blend.

Histogram equalization: `cv2.equalizeHist(gray)`. **Only 8-bit single-channel**. It spreads clumped intensities across 0–255 $\rightarrow$ better global contrast.

2.3 Lab 1 task solutions

Part 1: setup. `%pip install opencv-python` and `%pip install matplotlib` (note the % magic), then `import cv2, matplotlib.pyplot as plt; print(cv2.__version__)`.

Task 1: GroceryManager

```
class GroceryManager:
    def __init__(self):
        self.items = {} # name -> {'quantity', 'price'}
    def add_item(self, item, quantity, price):
        if item in self.items:
            self.items[item]['quantity'] += quantity
            self.items[item]['price'] = price
        else:
            self.items[item] = {'quantity': quantity, 'price': price}
    def remove_item(self, item):
        if item not in self.items:
            print(f"Error: '{item}' is not in the list"); return False
        del self.items[item]; return True
    def view_list(self):
        if not self.items: print('List is empty'); return
        for n, d in self.items.items():
            print(f"{n:<10} qty={d['quantity']:<3} price={d['price']:<7}
subtotal={d['quantity']*d['price']}")
    def calculate_total(self):
        return sum(d['quantity'] * d['price'] for d in self.items.values())
```

Task 2: nested-dict student records

```

students = {101: {'Name': 'Ayesha', 'Major': 'AI', 'Grades': [88, 92, 79]},
            102: {'Name': 'Bilal', 'Major': 'CS', 'Grades': [70, 65, 80]}}

def top_student(records):
    best, best_avg = None, float('-inf')
    for info in records.values():
        if not info['Grades']: continue
        avg = sum(info['Grades']) / len(info['Grades'])
        if avg > best_avg: best, best_avg = info['Name'], avg
    return best

def search_by_major(records, major):
    hits = [i['Name'] for i in records.values() if i['Major'].lower() ==
major.lower()]
    print(hits if hits else f'No students in {major}')

```

Task 3: exactly the skeleton in 1.1 plus `plt.figure(figsize=(8,6))`, `plt.axis('off')`, and `plt.title('...', fontsize=16, color='darkred', pad=15)`.

Task 4: target board on a blank canvas

```

canvas = np.zeros((800, 800, 3), np.uint8)          # black
cx, cy = canvas.shape[1] // 2, canvas.shape[0] // 2  # (400, 400)
for r, c in zip([350, 280, 210, 140, 70],
                [(255,255,255), (0,0,255)]*2 + [(255,255,255)]):
    cv2.circle(canvas, (cx, cy), r, c, -1)           # -1 = filled; BIGGEST first
cv2.rectangle(canvas, (cx-350, cy-350), (cx+350, cy+350), (0,255,0), 3)
print('Canvas centre:', (cx, cy))

```

Draw the largest circle first, otherwise it paints over the smaller ones. `thickness=-1` fills.

Task 5: heavy blur + centre ROI

```

blur = cv2.GaussianBlur(rgb, (25, 25), 0)
h, w = rgb.shape[:2]; cy, cx = h // 2, w // 2
roi_o = rgb[cy-150:cy+150, cx-150:cx+150]          # 300x300
roi_b = blur[cy-150:cy+150, cx-150:cx+150]
# two subplots, plt.axis('off'), plt.title(..., fontsize=14, color='navy')

```

Task 6: semi-transparent caption bar

```
h, w = rgb.shape[:2]
overlay = rgb.copy()
cv2.rectangle(overlay, (0, int(0.8*h)), (w, h), (0, 0, 255), -1) # blue on
an RGB array
out = cv2.addWeighted(overlay, 0.5, rgb, 0.5, 0)
cv2.putText(out, 'My Title', (30, int(0.92*h)), cv2.FONT_HERSHEY_SIMPLEX,
1.5, (255,255,255), 3)
```

Idea: draw the solid bar on a *copy*, then blend the copy with the original. Text goes on `out` (after blending) so it stays crisp.

Task 7: global vs adaptive, then rotate

```
gray = cv2.imread('doc.jpg', cv2.IMREAD_GRAYSCALE)
_, glob = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)
adap = cv2.adaptiveThreshold(gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                             cv2.THRESH_BINARY, 11, 2) # blockSize=11
(odd), C=2
h, w = adap.shape
M = cv2.getRotationMatrix2D((w/2, h/2), 45, 0.8) # scale 0.8 keeps
corners in
rot = cv2.warpAffine(adap, M, (w, h))
```

Adaptive thresholding computes a *different threshold per pixel* from its neighbourhood, so it survives uneven lighting (full details in Lab 5).

Task 8: masking with bitwise logic

```
a = cv2.resize(cv2.imread('a.jpg'), (500, 500)); b =
cv2.resize(cv2.imread('b.jpg'), (500, 500))
mask = np.zeros((500, 500), np.uint8)
cv2.circle(mask, (250, 250), 150, 255, -1) # FILLED white shape
(thickness -1)
fg = cv2.bitwise_and(a, a, mask=mask) # shape cut out of image A
bg = cv2.bitwise_and(b, b, mask=cv2.bitwise_not(mask)) # everything except
shape, from B
out = cv2.bitwise_or(fg, bg)
```

`bitwise_and(src1, src2, mask=m)` keeps pixels only where `m != 0`. A "thick" outline (not filled) would only cut a ring, so fill it.

Task 9: pandas profile

```
rgb = cv2.cvtColor(cv2.imread('img.jpg'), cv2.COLOR_BGR2RGB)
df = pd.DataFrame({'Red': rgb[:, :, 0].ravel(), 'Green': rgb[:, :, 1].ravel(),
                  'Blue': rgb[:, :, 2].ravel()})
print(df.describe())
```

`.ravel()` / `.flatten()` / `.reshape(-1)` turn a 2D channel into a 1D array.

2.4 Lab 1 quick self-test

- Why does a photo look blue-tinted in Matplotlib when read with OpenCV? (*BGR vs RGB.*)
- What does `cv2.imread` return for a wrong path? (*None.*)
- Output size of `cv2.resize(img, (200, 100))`? (*width 200, height 100 → shape (100, 200, 3).*)
- What is `255 + 10` in NumPy `uint8` vs `cv2.add`? (*9 vs 255.*)

3. Lab 2: Photometric (point) transforms, histograms, colour, fusion, video

⚠ The Lab 2 manual was missing, so this section = the Lab 2 task sheet + standard textbook material (Gonzalez & Woods-style point processing). Your teacher said to study **Log, Gamma, Piecewise, Histogram**. Those are covered in depth below.

3.1 The big idea: point transformations

A **point (intensity) transformation** maps each pixel's input intensity r to an output intensity s using a function T , **independently of its neighbours and position**:

$$s = T(r), \text{ for 8-bit images } r, s \in [0, 255] \text{ (} L = 256 \text{ levels)}$$

This is a **photometric** transform: it changes *what a pixel looks like*, not *where it is* (Lab 3 geometric transforms do the opposite).

Transform	Formula	Effect	Typical use
Negative	$s = 255 - r$	inverts brightness	white-on-dark detail in dark regions
Log	$s = c \cdot \log(1 + r)$	expands dark values, compresses bright	underexposed images, huge dynamic range
Inverse log	$s = \exp(r/c) - 1$	opposite of log	rarely used

Transform	Formula	Effect	Typical use
Power-law (gamma)	$s = c \cdot r^\gamma$	$\gamma < 1$ brightens (expands darks); $\gamma > 1$ darkens (expands brights)	display correction, midtone tuning
Contrast stretch	$s = (r - r_{\min}) / (r_{\max} - r_{\min}) \cdot 255$	uses the full range	low-contrast images
Piecewise linear	different slope per range	custom contrast control	targeted enhancement
Histogram equalization	$s = 255 \cdot \text{CDF}(r)$	flattens the histogram	automatic global contrast

3.2 Log transform (study this one properly)

$s = c \cdot \log(1 + r)$. The **+1** avoids $\log(0)$. Choose $c = 255 / \log(1 + r_{\max})$ so the brightest input maps exactly to 255.

Why it works: the log curve is steep near 0 and flat near 255, so a *narrow band of dark values* gets stretched over a wide output range, while bright values get squeezed. That's why it reveals the faint outer ribcage in an underexposed X-ray.

```
f = img.astype(np.float32)          # CAST FIRST: uint8 + 1 overflows at 255
c = 255 / np.log(1 + f.max())
log_img = (c * np.log(1 + f)).astype(np.uint8)
```

Trap: `np.log(1 + img)` on a `uint8` array wraps 255+1 to 0. Always convert to float before adding. **Limit:** too much log washes out bright regions (bones become grey).

3.3 Power-law (gamma) transform

Normalised form: $s = 255 \cdot (r/255)^\gamma$ (the constant c is absorbed by normalising).

γ	Curve	Result
$\gamma < 1$ (e.g. 0.4)	above the identity line	image brighter , dark detail expanded (like log, but tunable)
$\gamma = 1$	identity	unchanged

γ	Curve	Result
$\gamma > 1$ (e.g. 2.2)	below the identity line	image darker , bright detail expanded

```
gamma = 0.5
lut = np.array([(i / 255.0) ** gamma * 255 for i in range(256)],
dtype=np.uint8)
out = cv2.LUT(gray, lut) # fast: table lookup per pixel
```

`cv2.LUT(src, lut)` replaces every pixel value v by `lut[v]`. Any point transform can be done as a 256-entry lookup table.

Log vs gamma: log has one fixed shape; gamma has a knob (γ). Fractional γ (< 1) is how the task “fine-tunes midtones” so soft lung tissue is clearer while the bones’ harsh contrast is reduced. Many monitors are roughly $\gamma \approx 2.2$ devices, and *gamma correction* pre-applies the inverse to compensate (I believe that’s the standard explanation; verify with your slides).

3.4 Piecewise-linear transform & contrast stretching

Split the input range at control points (r_1, s_1) and (r_2, s_2) ; each segment has its own slope:

- segment 1: $0 \rightarrow r_1$ maps to $0 \rightarrow s_1$ (slope s_1/r_1)
- segment 2: $r_1 \rightarrow r_2$ maps to $s_1 \rightarrow s_2$ (slope $(s_2-s_1)/(r_2-r_1)$)
- segment 3: $r_2 \rightarrow 255$ maps to $s_2 \rightarrow 255$ (slope $(255-s_2)/(255-r_2)$)

Slope > 1 stretches (more contrast) that range; slope < 1 compresses it. Special cases: $r_1 = s_1$ and $r_2 = s_2 \rightarrow$ identity; $r_1 = r_2$ with $s_1 = 0, s_2 = 255 \rightarrow$ **thresholding**; $(r_1, s_1) = (r_{\min}, 0), (r_2, s_2) = (r_{\max}, 255) \rightarrow$ **contrast stretching**.

```
r1, s1, r2, s2 = 70, 20, 180, 240
lut = np.interp(np.arange(256), [0, r1, r2, 255], [0, s1, s2,
255]).astype(np.uint8)
out = cv2.LUT(gray, lut)
# min-max stretch shortcut:
stretch = cv2.normalize(gray, None, 0, 255, cv2.NORM_MINMAX)
```

(Related, in case it’s asked: **intensity-level slicing** highlights one intensity band; **bit-plane slicing** shows each of the 8 bits as an image.)

3.5 Histograms

The **histogram** $h(r_k) = n_k$ counts pixels at each intensity r_k . The **normalised** histogram $p(r_k) = n_k / (M \cdot N)$ behaves like a probability distribution.

```
h = cv2.calcHist([gray], [0], None, [256], [0, 256]) # images, channels,
mask, bins, ranges
plt.hist(gray.ravel(), bins=256, range=(0, 256)) # or plt.plot(h)
```

`calcHist` argument order: `[images], [channel], mask, [bins], [min, max]`. The image goes **inside a list**.

Histogram looks like	Image is
bunched at the left	dark / underexposed
bunched at the right	bright / washed out
narrow in the middle	low contrast
spread over the full range	good contrast

3.6 Histogram equalization (full mechanism)

Goal: make the output histogram roughly flat so all intensities are used.

1. Compute the histogram n_k and $p(r_k) = n_k / (M \cdot N)$.
2. Compute the cumulative distribution $\mathbf{CDF(k)} = \sum_{j \leq k} p(r_j)$.
3. Map every pixel: $\mathbf{s_k = round((L - 1) \cdot CDF(k))}$.

Worked mini-example (4 gray levels, 16 pixels, $L = 4$): counts $n = [6, 5, 3, 2] \rightarrow p = [0.375, 0.3125, 0.1875, 0.125] \rightarrow \mathbf{CDF} = [0.375, 0.6875, 0.875, 1.0] \rightarrow (L-1) \cdot \mathbf{CDF} = [1.125, 2.06, 2.625, 3] \rightarrow$ rounded $s = [1, 2, 3, 3]$. So level $0 \rightarrow 1, 1 \rightarrow 2, 2 \rightarrow 3, 3 \rightarrow 3$. (OpenCV's implementation, I believe, subtracts the smallest non-zero CDF value first, so exact numbers can differ slightly; use whichever your class formula is.)

```
eq = cv2.equalizeHist(gray) # 8-bit single
channel ONLY
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
eq_local = clahe.apply(gray) # adaptive, limits
noise amplification
```

- **Limits:** global method; can over-amplify noise and wash out images with a large uniform area. **CLAHE** equalizes per tile with a clip limit.
- **Colour images:** don't equalize B, G, R separately (colours shift). Convert to YCrCb (or HSV) and equalize only Y (or V).

3.7 Colour mapping (false colour / heatmap)

```
heat = cv2.applyColorMap(eq, cv2.COLORMAP_JET)    # input: 8-bit gray; output:
3-channel BGR
plt.imshow(cv2.cvtColor(heat, cv2.COLOR_BGR2RGB))
```

JET maps low intensity → blue, mid → green/yellow, high → red. **Why it helps:** the eye separates many hues far better than many near-identical gray levels, so subtle intensity boundaries (e.g. fluid edges) pop. It adds **no new information**; it just re-codes intensity as colour.

3.8 Colour balance

Corrects a colour cast by adjusting each channel. Two simple models:

```
f = bgr.astype(np.float32)
means = f.reshape(-1, 3).mean(axis=0)          # [B, G, R] means
# (a) Shift (additive): move every channel mean to the overall mean
shifted = np.clip(f + (means.mean() - means), 0, 255).astype(np.uint8)
# (b) Gray-world gain (multiplicative): scale channels to equal means
gained = np.clip(f * (means.mean() / means), 0, 255).astype(np.uint8)
```

The task wording ("color balance shift") fits (a); say which you used and why. **Gray-world assumption:** the average scene colour should be neutral gray.

3.9 Thresholding to isolate dense tissue

```
_, dense = cv2.threshold(eq, 200, 255, cv2.THRESH_BINARY)    # keep only
very bright (bone, dense fluid)
only_dense = cv2.bitwise_and(eq, eq, mask=dense)             # original
intensities where mask is white
```

Threshold choice is judged visually or by Otsu (Lab 5). Say why in one line ("dense tissue attenuates X-rays → high intensity").

3.10 Task 1 pipeline: chest X-ray enhancement

```
xr = cv2.imread('data/sample_xray.png', cv2.IMREAD_GRAYSCALE)    # relative
path!
if xr is None: raise SystemExit('image not found')
eq    = cv2.equalizeHist(xr)
heat  = cv2.applyColorMap(eq, cv2.COLORMAP_JET)
f = heat.astype(np.float32); m = f.reshape(-1,3).mean(0)
bal    = np.clip(f + (m.mean() - m), 0, 255).astype(np.uint8)
_, dense = cv2.threshold(eq, 200, 255, cv2.THRESH_BINARY)
log_x = (255/np.log(1+xr.max())) * np.log(1 +
xr.astype(np.float32)).astype(np.uint8)
gam_x = cv2.LUT(xr, np.array([(i/255)**0.6*255 for i in range(256)],
np.uint8))    # gamma 0.6
# plot: raw | equalized | heatmap | balanced | dense mask | log | gamma, then
cv2.imwrite('output/...')
```

Document for each step: what became visible (lung fields, ribs, fluid boundaries) and why.

3.11 Task 2: CT + MRI fusion

```
ct  = cv2.imread('data/ct.png', 0); mri = cv2.imread('data/mri.png', 0)
mri = cv2.resize(mri, (ct.shape[1], ct.shape[0]))    # SAME size is
mandatory for addWeighted
ct_eq, mri_eq = cv2.equalizeHist(ct), cv2.equalizeHist(mri)
ct_c  = cv2.cvtColor(ct_eq, cv2.COLOR_GRAY2BGR)    # CT: gray
structure, 3-channel
mri_c = cv2.applyColorMap(mri_eq, cv2.COLORMAP_JET)    # MRI: heatmap for
soft tissue
fused = cv2.addWeighted(ct_c, 0.7, mri_c, 0.3, 0)    # heavier alpha on
CT = sharp edges
f = fused.astype(np.float32)
logf  = 255/np.log(1+f.max()) * np.log(1+f)    # lift crushed
blacks
final = (255 * (logf/255) ** 1.2).astype(np.uint8)    # gamma > 1 tames
what log blew out
# plot CT | MRI | fused with plt.subplot(1, 3, i)
```

Weighting logic: $\text{addWeighted}(\text{src1}, \alpha, \text{src2}, \beta, \gamma) = \alpha \cdot \text{src1} + \beta \cdot \text{src2} + \gamma$. Keep $\alpha + \beta = 1$ to preserve overall brightness; $\alpha > \beta$ favours CT edges, β gives MRI tissue tint. This only makes sense if the slices are **already spatially aligned** (the task says “perfectly corresponding”); otherwise you’d need registration (homography/affine, Lab 3).

3.12 Task 3: real-time echo video

```
cap = cv2.VideoCapture('data/echo.mp4')           # file path, not 0
(0 = webcam)
if not cap.isOpened(): raise SystemExit('cannot open video')
while True:
    ok, frame = cap.read()                         # ok=False at end of
video
    if not ok: break
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    eq = cv2.equalizeHist(gray)
    heat = cv2.applyColorMap(eq, cv2.COLORMAP_JET)
    f = heat.astype(np.float32); m = f.reshape(-1,3).mean(0)
    f = np.clip(f + (m.mean() - m), 0, 255)        # colour balance
    f = 255/np.log(1+f.max()) * np.log(1+f)        # log: reveal dark
chambers
    f = 255 * (f/255) ** 1.5                       # power-law gamma>1:
suppress bright backscatter
    out = f.astype(np.uint8)
    cv2.imshow('Raw | Enhanced', np.hstack([frame, out])) # same height,
same channels required
    if cv2.waitKey(25) & 0xFF == ord('q'): break
cap.release(); cv2.destroyAllWindows()
```

- `cv2.waitKey(25)` waits 25 ms per frame (≈ 40 fps cap); `0` would freeze on every frame.
- `np.hstack` needs equal height and equal channel count, so enhanced output must be 3-channel like `frame`.
- Always `release()` the capture. In a notebook, `cv2.imshow` is unreliable, so show frames with Matplotlib or write a file with `cv2.VideoWriter`.
- My observation: normalising by `f.max()` per frame can make brightness flicker between frames; a fixed constant is steadier (try it).

3.13 Lab 2 submission rules (from the task sheet)

- Repo layout: `Task_1_Chest_XRay/`, `Task_2_Cardiac_Fusion/`, `Task_3_Echo_Analysis/`, each with `data/`, `output/`, a notebook, and `README.md`; plus root `requirements.txt` and `README.md`.
- **100 MB rule:** GitHub rejects large files, so upload only the sample images/clips your code actually uses.
- **Relative paths** only (`data/sample_xray.png`), never `C:/Users/...`.
`requirements.txt` pins versions (e.g. `opencv-python==4.8.0`).

- Datasets named: COVID19+PNEUMONIA+NORMAL chest X-rays (Kaggle), Heart CT & MRI (Kaggle), Stanford EchoNet-Dynamic (Kaggle).

3.14 Lab 2 self-test

- What does $\gamma = 0.5$ do? (*Brightens; expands dark range.*) What does $\gamma = 2$ do? (*Darkens; expands bright range.*)
- Which gets used on a histogram bunched at the far left: log, gamma < 1 , equalization? (*All three help; gamma < 1 and log lift darks, equalization redistributes.*)
- Why must you cast to float before log? (*uint8 overflow.*)
- Why equalize CT and MRI before blending? (*Maximise each one's dynamic range so neither dominates by accident.*)

4. Lab 3: Geometric transformations

4.1 Geometry vs photometry

- **Geometric:** changes *where* a pixel is: $(x, y) \rightarrow (x', y')$. Pixel values themselves are carried along (then interpolated).
- **Photometric:** changes *what* a pixel looks like (brightness, contrast, gamma, colour): $I(x, y) \rightarrow I'(x, y)$.

Image coordinates: origin at the **top-left**, **x $\rightarrow$ right**, **y $\downarrow$ down**. For a 640×480 image the corners are (0,0), (639,0), (0,479), (639,479). Indexing is `image[y, x]`, but points are written (x, y).

Matrix sizes: 2D affine = 2×3 (`warpAffine`); 2D in homogeneous coordinates / projective = 3×3 (`warpPerspective`); 3D homogeneous = 4×4 .

4.2 Reading a 2×2 matrix (the Lab 3 mental model)

$[[a, b], [c, d]]$ acts on a point as $x' = a \cdot x + b \cdot y$, $y' = c \cdot x + d \cdot y$.

Read it by COLUMNS: column 1 (a, c) is where the x-axis unit vector (1, 0) lands; column 2 (b, d) is where the y-axis unit vector (0, 1) lands. Every other point follows its axes.

Entry	Controls
a	horizontal scale (x of new x-axis)
d	vertical scale (y of new y-axis)
b	horizontal shear (x of new y-axis)

Entry	Controls
c	vertical shear (y of new x-axis)

Negative a or d $\Rightarrow$ reflection. A *pure rotation* needs all four entries together: $[[\cos\theta, -\sin\theta], [\sin\theta, \cos\theta]]$.

4.3 Linear transformations ($p' = A \cdot p$)

Type	Matrix	Equation
Scaling	$[[sx, 0], [0, sy]]$	$x' = sx \cdot x, y' = sy \cdot y$
Rotation	$[[\cos\theta, -\sin\theta], [\sin\theta, \cos\theta]]$	rotates about the origin
Reflection (across x-axis)	$[[1, 0], [0, -1]]$	$y' = -y$
Horizontal shear	$[[1, k], [0, 1]]$	$x' = x + k \cdot y, y' = y$

Key limitation: $A \cdot 0 = 0$. A linear map **always keeps the origin fixed**, so it cannot translate. (Manual example: you can't move (10, 20) to (110, 70) with a 2×2 matrix.)

4.4 Affine = linear + translation; homogeneous coordinates

Affine: $p' = A \cdot p + b$ (A is 2×2 , b is the translation vector). It **preserves collinearity, parallelism, and ratios of distances along a line**, but not angles or lengths in general.

Homogeneous trick: write points as $(x, y, 1)$. Then translation becomes matrix multiplication:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & tx \\ 0 & 1 & ty \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad \Rightarrow \quad x' = x + tx, \quad y' = y + ty$$

Why bother: rotation, scaling, shear **and** translation all become 3×3 matrix products, so they can be multiplied into **one master matrix** and applied in a single pass (GPUs are optimised for chains of matrix multiplications).

Composition order (very testable): with column vectors, $M = T \cdot R \cdot S$ applies **S first, then R, then T** (right-to-left). Matrix multiplication is **not commutative**: rotate-then-translate $\neq$ translate-then-rotate.

4.5 The hierarchy (know this table)

Transform	Degrees of freedom	What it preserves	OpenCV
Translation	2	orientation, size, shape (everything but position)	<code>warpAffine</code>
Rigid (rotation + translation)	3	lengths, angles, shape	<code>warpAffine</code>
Similarity (rigid + uniform scale)	4	angles, ratios of lengths	<code>warpAffine</code>
Affine	6	parallelism, collinearity, length ratios along a line	<code>getAffineTransform</code> , <code>warpAffine</code>
Projective / homography	8	collinearity (straight lines stay straight) only; parallelism is lost	<code>getPerspectiveTransform</code> , <code>warpPerspective</code>

(Standard textbook values, e.g. Szeliski; I'm confident but verify with your slides.) Each class contains the one above it: rigid $\subset$ similarity $\subset$ affine $\subset$ projective.

⚠ The manual's tree diagram puts Similarity and Rigid *under* Scale, and its bullet list indents "translation" under "perspective". Both look like layout mistakes (translation is affine, not linear). If an MCQ asks, remember: **translation = affine but NOT linear**.

4.6 Projective transformations (homography)

3×3 matrix H (h33 is usually normalised to 1, leaving **8 unknowns** → **4 point pairs**):

$$x' = (h_{11} x + h_{12} y + h_{13}) / (h_{31} x + h_{32} y + h_{33})$$

$$y' = (h_{21} x + h_{22} y + h_{23}) / (h_{31} x + h_{32} y + h_{33})$$

The denominator is the whole story. In affine maps the bottom row is [0, 0, 1], so the denominator = 1 (uniform scaling everywhere). In projective maps it varies with (x, y): points "further away" get a larger denominator and shrink → parallel lines converge to a **vanishing point**. Used for: perspective correction of documents (trapezoid → rectangle), camera viewpoint changes, stitching panoramas.

4.7 OpenCV function cheat-sheet for Lab 3

Goal	Code
Rotation matrix (2×3)	<code>M = cv2.getRotationMatrix2D((cx, cy), angle, scale)</code> (+angle = counter-clockwise)
Apply affine	<code>cv2.warpAffine(img, M, (w, h))</code> (dsize = width, height)
Affine from 3 point pairs	<code>M = cv2.getAffineTransform(src3, dst3)</code> (float32, shape (3,2))
Invert an affine	<code>cv2.invertAffineTransform(M)</code>
Homography from 4 pairs	<code>H = cv2.getPerspectiveTransform(src4, dst4)</code>
Homography from many pairs	<code>H, inl = cv2.findHomography(src, dst, cv2.RANSAC, 5.0)</code>
Apply homography	<code>cv2.warpPerspective(img, H, (w, h))</code>
Scale	<code>cv2.resize(img, None, fx=3, fy=3, interpolation=cv2.INTER_LINEAR)</code>

Points must be `np.float32`. Pixels with no source come out **black** by default (`borderMode` / `borderValue` change that).

4.8 The class notebook decoded

- **Rotation:** `center=(width//2, height//2)`, angle 45, scale 1.0 → corners cut, black fill.
- **Scaling:** `cv2.resize(image, None, fx=3, fy=3, interpolation=cv2.INTER_LINEAR)`. It looks the same size in the plot because Matplotlib auto-fits, so check `.shape`.
- **Shear (x):** `M = np.float32([[1, kx, 0], [0, 1, 0]])` with `kx = 0.5`; canvas widened to `new_width = int(width + abs(kx)*height)` so nothing is clipped.
- **Translation:** `M = [[1,0,tx],[0,1,ty]]`, `tx = 100, ty = 50` → content moves **right 100, down 50**; black strip appears at top/left.
- **Rigid:** `getRotationMatrix2D(center, 30, scale=1.0)` then `M[0,2] += 100; M[1,2] += 50` (scale **must** be 1.0 for rigid).
- **Perspective:** `src = [[0,0],[w-1,0],[w-1,h-1],[0,h-1]]` and four shifted `dst` corners → `getPerspectiveTransform` → `warpPerspective`.

4.9 Lab 3 tasks: solutions and the reasoning

Task 1: 3× enlargement, kept centred (linear scale)

```
h, w = img.shape[:2]; cx, cy = w/2, h/2
M = np.float32([[3, 0, cx - 3*cx],          # tx = cx - 3cx = -2cx (so the
               [0, 3, cy - 3*cy]])          # centre stays put)
zoom = cv2.warpAffine(img, M, (w, h))      # same canvas: you see the central
third, enlarged
```

Derivation: want the centre fixed: $3 \cdot cx + tx = cx \Rightarrow tx = -2cx$. (Alternatively use a (3w, 3h) canvas with a pure scale matrix to see everything.)

Task 2: undo a 45° rotation without chopping corners

```
ang = 45; t = np.deg2rad(ang)
new_w = int(h*abs(np.sin(t)) + w*abs(np.cos(t)))
new_h = int(h*abs(np.cos(t)) + w*abs(np.sin(t)))
R = np.array([[np.cos(t), -np.sin(t)], [np.sin(t), np.cos(t)]]) # manual
2x2 (sine & cosine)
shift = np.array([new_w/2, new_h/2]) - R @ np.array([w/2, h/2]) # re-
centre on the bigger canvas
M = np.hstack([R, shift[:, None]]).astype(np.float32)
fixed = cv2.warpAffine(img, M, (new_w, new_h))
```

Bounding-box formulas: $\text{new_w} = h|\sin\theta| + w|\cos\theta|$, $\text{new_h} = h|\cos\theta| + w|\sin\theta|$. **Sign warning:** because y points down, this matrix with $+\theta$ turns the picture *clockwise* on screen, while `getRotationMatrix2D(+θ)` turns it *counter-clockwise*. If the result gets worse, flip the sign.

Task 3: un-slant a barcode (shear)

```
k = np.tan(np.deg2rad(slant_deg))          # k = tan(slant angle); try both
signs
M = np.float32([[1, k, 0], [0, 1, 0]])      # x' = x + k*y
fixed = cv2.warpAffine(img, M, (int(w + abs(k)*h), h))
```

Each row y is pushed horizontally by $k \cdot y$, so slanted bars become vertical. If $k < 0$, shift by $+\text{abs}(k) \cdot h$ in the translation slot so nothing falls off the left edge.

Task 4: slide the map back (translation) `M = np.float32([[1, 0, 150], [0, 1, 80]])`. The X is 150 px *left* and 80 px *up* off-screen, so bring it back by moving content **right +150**, **down +80**. Need `warpAffine(img, M, (w + 150, h + 80))` if you want to avoid clipping the far edges.

Task 5–6: rigid and similarity via ONE 3×3 matrix

```

def T(tx, ty): return np.array([[1,0,tx],[0,1,ty],[0,0,1]], np.float32)
def Rot(deg, cx=0, cy=0):                                     # rotation about (cx, cy)
    t = np.deg2rad(deg); c, s = np.cos(t), np.sin(t)
    return T(cx, cy) @ np.array([[c,-s,0],[s,c,0],[0,0,1]], np.float32) @ T(-
cx, -cy)
def Sc(s): return np.diag([s, s, 1]).astype(np.float32)

M_rigid = T(tx, ty) @ Rot(theta, cx, cy)                    # Task 5: rotate, then
translate
M_sim   = T(tx, ty) @ Rot(theta, 0, 0) @ Sc(s)              # Task 6: scale, rotate,
translate in one matrix
out = cv2.warpAffine(img, M_sim[:2], (w, h))                # warpAffine wants the top
2 rows

```

Rigid keeps lengths and angles (shape unchanged). Similarity keeps angles but not size. Both are special affine cases.

Task 7: general affine from 3 landmark pairs (solve 6 unknowns)

```

src = np.float32([[x1,y1],[x2,y2],[x3,y3]])                # landmarks in the glitched
image
dst = np.float32([[X1,Y1],[X2,Y2],[X3,Y3]])                # where they should be
A = np.hstack([src, np.ones((3,1))])                        # rows [x y 1]
M = np.linalg.solve(A, dst).T                                # 2x3 = [[a,b,tx],[c,d,ty]]
fixed = cv2.warpAffine(glitched, M, (w, h))                 # same as
cv2.getAffineTransform(src, dst)

```

Each point gives $x' = a \cdot x + b \cdot y + tx$ and $y' = c \cdot x + d \cdot y + ty$. Three points $\rightarrow$ three equations per output coordinate $\rightarrow$ two 3×3 systems $\rightarrow$ 6 unknowns. Direction matters: M must map *glitched* $\rightarrow$ *original*. If you were given original $\rightarrow$ glitched, use

`cv2.invertAffineTransform(M)`. Why 3 points: affine has 6 DoF and each point supplies 2 equations.

Task 8: sidewalk chalk art $\rightarrow$ bird's-eye view

```

src = np.float32([[x_tl,y_tl],[x_tr,y_tr],[x_br,y_br],[x_bl,y_bl]]) #
clicked corners, consistent order
S = 500
dst = np.float32([[0,0],[S,0],[S,S],[0,S]])                  # a
perfect square
H = cv2.getPerspectiveTransform(src, dst)
top = cv2.warpPerspective(img, H, (S, S))

```

Keep the corner order identical in `src` and `dst`. Four points $\rightarrow$ 8 equations $\rightarrow$ 8 unknowns of H.

Task 9: stadium panorama (homography stitching)

```
# pts2: 4 points in image 2; pts1: the SAME 4 physical points in image 1
(float32, shape (4,2))
H = cv2.getPerspectiveTransform(pts2, pts1)          # maps image-2 coords ->
image-1 coords
pano = cv2.warpPerspective(img2, H, (w1 + w2, max(h1, h2))) # wide canvas
pano[0:h1, 0:w1] = img1                                # paste reference image on
top
```

With more than 4 matches use `cv2.findHomography(pts2, pts1, cv2.RANSAC, 5.0)`. If image 2 is to the *left*, warp image 1 into image 2's frame instead (or add a translation so negative coordinates don't get cut). Lab 6 automates the point picking with SIFT.

Task 10: master forger (full hierarchy)

```
# 1) linear scale  2) rigid move  3) projective fit into the angled frame
S3 = np.diag([0.5, 0.5, 1]).astype(np.float32)
Rg = T(tx, ty) @ Rot(angle, 0, 0)                # rigid
ph, pw = art.shape[:2]
corners = np.float32([[0,0],[pw,0],[pw,ph],[0,ph]]).reshape(-1,1,2)
near = cv2.perspectiveTransform(corners, Rg @ S3) # painting corners
after scale+rigid
frame_pts = np.float32([[fx1,fy1],[fx2,fy2],[fx3,fy3],[fx4,fy4]]) # frame's
4 inner corners
H = cv2.getPerspectiveTransform(near.reshape(4,2), frame_pts)
Htot = H @ Rg @ S3                                # compose once: avoids
repeated interpolation
warped = cv2.warpPerspective(art, Htot, (fw, fh))
mask = cv2.warpPerspective(np.full((ph, pw), 255, np.uint8), Htot, (fw, fh))
frame[mask > 0] = warped[mask > 0]                # paste only inside
the warped quad
```

The lesson: climb the hierarchy (linear $\rightarrow$ rigid $\rightarrow$ projective); the final projective step is the only one that creates perspective convergence.

4.10 Lab 3 self-test

- Why can't a 2×2 matrix translate? ($A \cdot 0 = 0$, origin fixed.)
- How many point pairs for affine / projective? (3 / 4.)
- What does the bottom row $[0, 0, 1]$ mean? (Affine: denominator = 1; no perspective.)
- What's the effect of `getRotationMatrix2D(center, 45, 0.8)` on corners? (Rotated and shrunk to 80%, so corners don't get cut.)
- Does `cv2.warpAffine` take (h, w) or (w, h)? ((w, h).)

5. Lab 4: Feature extraction, filtering/convolution, edge detection

5.1 What is a feature?

Feature extraction turns raw, high-dimensional pixels into a compact, meaningful representation that keeps what matters and drops what doesn't (reduces dimensionality, enhances relevant info, cuts computation).

- **Local features:** from specific regions (keypoints, corners, small patches). Used for matching and detection.
- **Global features:** computed over the whole image (colour histograms, texture descriptors, image moments).
- **Techniques:** histogram-based, filter-based (Gabor filters, Haar wavelets), deep-learning features (CNN layers learn them from data).
- **Applications:** object recognition, face recognition, gesture recognition, medical imaging, autonomous vehicles. **Challenges:** lighting, scale, orientation, noise.

5.2 HOG: Histogram of Oriented Gradients

Idea: describe local shape by the *distribution of gradient directions*. Good for object/pedestrian detection.

Pipeline (manual order):

1. Convert to grayscale.
2. Compute gradient magnitude + orientation per pixel (Sobel; I believe the original paper used a simple $[-1, 0, 1]$ derivative).
3. Divide into small **non-overlapping cells** (e.g. 8×8 px).
4. Per cell, build a histogram of gradient orientations (e.g. 9 bins over $0-180^\circ$, 20° each), each pixel voting with its gradient magnitude.
5. **Block normalisation:** group cells into blocks (e.g. 2×2 cells, blocks overlap with a stride of one cell) and normalise to reduce lighting effects.
6. Concatenate all normalised block histograms → the **feature vector**.

```
from skimage.feature import hog
from skimage import exposure
gray = cv2.imread('image.jpg', cv2.IMREAD_GRAYSCALE)
features, hog_img = hog(gray, pixels_per_cell=(8, 8), cells_per_block=(2, 2),
visualize=True)
hog_vis = exposure.rescale_intensity(hog_img, in_range=(0, 10)) # brighten
for display
```

Vector length (testable): with 9 orientations, $\text{length} = (\text{cells}_y - 1) \cdot (\text{cells}_x - 1) \cdot 2 \cdot 2 \cdot 9$.

Example: a 64×128 window $\rightarrow 8 \times 16$ cells $\rightarrow 7 \times 15 = 105$ blocks $\rightarrow 105 \cdot 36 = 3780$.

5.3 LBP: Local Binary Pattern (texture)

Idea: compare each pixel to its neighbours. Neighbour $\geq$ centre $\rightarrow 1$, else 0; the bits form a number; the **histogram of these numbers** over the image is the texture descriptor.

Worked example (centre = 6, neighbours clockwise from top-left):

```
5 2 1
7 6 8    5,2,1,8,4,3,9,7  ->  0,0,0,1,0,0,1,1  ->  00010011  =  19
9 3 4
```

(Starting point and direction are conventions; stay consistent.)

```
from skimage import feature
radius = 1; n_points = 8 * radius # P = 8*R neighbours
lbp = feature.local_binary_pattern(gray, n_points, radius, method='uniform')
hist, _ = np.histogram(lbp.ravel(), bins=np.arange(0, n_points + 3), range=(0, n_points + 2))
hist = hist.astype('float'); hist /= (hist.sum() + 1e-6) # normalise (1e-6 avoids divide-by-zero)
```

- **radius** = circle radius; **n_points** = samples on the circle.
- **'uniform'**: a pattern is uniform if it has ≤ 2 circular 0 $\leftrightarrow$ 1 transitions. There are $P+1$ uniform patterns plus one catch-all bin for non-uniform ones $\rightarrow$ **$P + 2$ bins** (10 for $P = 8$), which is why `bins=np.arange(0, n_points+3)`. Without `uniform` you get $2^8 = 256$ patterns.
- **Strengths:** cheap; robust to monotonic lighting changes (only comparisons, not absolute values). **Weaknesses:** noise-sensitive in flat regions; local only.

5.4 Colour histogram

Counts pixels per colour bin (RGB or HSV).

```
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
h3 = cv2.calcHist([hsv], [0, 1, 2], None, [8, 8, 8], [0, 180, 0, 256, 0, 256])
vec = cv2.normalize(h3, h3).flatten() # 8*8*8 = 512-dim
feature
```

OpenCV hue range is 0–179 (not 0–359), S and V are 0–255. Wrong `ranges` is a classic bug.

5.5 HED: Histogram of Edge Directions

Pipeline: grayscale → (optional) Gaussian smoothing → edge detection (Canny/Sobel/Scharr) → orientation per pixel via arctan → bin into e.g. **8 bins of 45°** → count → normalise → feature vector (length 8).

```
sm = cv2.GaussianBlur(gray, (5, 5), 0)
edges = cv2.Canny(sm, 30, 70)
gx = cv2.Sobel(sm, cv2.CV_64F, 1, 0, ksize=3); gy = cv2.Sobel(sm, cv2.CV_64F, 0, 1, ksize=3)
ang = (np.arctan2(gy, gx) * 180 / np.pi) % 360 # map to [0, 360)
hist, bins = np.histogram(ang[edges > 0], bins=8, range=(0, 360)) # only edge pixels
```

⚠ The manual's code uses `np.arctan2(...)*180/π` directly, which returns **-180° to +180°**, then histograms over `range=(0, 360)`, so every negative angle is silently dropped. It also histograms *all* pixels, not just edge pixels. My version above fixes both (`% 360`, `ang[edges > 0]`). In an exam, mention the intent: "orientation histogram of edge pixels".

5.6 HIG: Histogram of Intensity Gradients

Same idea as HOG but **global** (no cells/blocks): Sobel dx, dy → magnitude $\sqrt{dx^2+dy^2}$ and orientation $\text{atan2}(dy, dx)$ → histogram of orientations (manual: 9 bins over 0–180°) → normalise by the sum. ⚠ Same angle-range caveat as above, and the manual's `np.histogram` doesn't use `weights=magnitude`, so each pixel counts equally (HOG weights by magnitude). Add `weights=magnitude.ravel()` if you want a magnitude-weighted histogram.

5.7 Texture energy and contrast

- **Energy** (uniformity/complexity): sum of squared pixel values in a local window (3×3 or 5×5). Higher → more intense/complex (per the manual).
- **Contrast**: standard deviation of intensities in the window: higher → more variation.

```
f = gray.astype(np.float32) # CAST: uint8**2 overflows!
k = (3, 3)
energy = cv2.boxFilter(f * f, -1, k, normalize=False) # local SUM of squares
mean = cv2.blur(f, k); mean_sq = cv2.blur(f * f, k)
contrast = np.sqrt(np.maximum(mean_sq - mean**2, 0)) # local std = sqrt(E[x^2] - E[x]^2)
```

⚠ The manual does `image ** 2` on a `uint8` array (wraps modulo 256) and computes "contrast" as `filter2D(image, -1, ones)`, which is just a local **sum**, not a standard deviation. The corrected version is above.

Lab 4 written task: texture analysis for wood / metal / fabric (model answer):

- **Technique 1: LBP.** Compares each pixel to its circular neighbours → histogram of micro-patterns. Suits fine, repetitive textures (fabric weave, brushed metal). *Advantages:* cheap, lighting-robust, compact. *Limits:* noise-sensitive, ignores colour and large structures, small support.
- **Technique 2: HOG / HED (orientation histograms) or Gabor filters.** Capture *directional* structure: wood grain has a dominant orientation; woven fabric has two. *Advantages:* captures shape/direction. *Limits:* not rotation-invariant unless handled, depends on cell size and image scale.
- **Technique 3 (optional): energy/contrast statistics.** Very fast, but weakly discriminative alone.
- **Pipeline:** gray → (denoise) → features (concatenate LBP + orientation histograms) → classifier (SVM / k-NN). Evaluate on held-out images; lighting and scale changes are the main risks.

5.8 Filtering and convolution

A **filter/kernel** is a small matrix of weights. At each pixel, the kernel is slid over the image, multiplied element-wise with the underlying patch, and **summed** (a dot product) to give the output pixel.

Kernel	Definition	Use
Box blur	<code>np.ones((3,3), np.float32) / 9</code>	simple smoothing (mean)
Gaussian	weights from a Gaussian	smoother, preferred for noise reduction
Sobel	<code>[[-1,0,1],[-2,0,2],[-1,0,1]]</code> (Gx), transposed for Gy	edges
Scharr	<code>cv2.Scharr(img, cv2.CV_64F, 1, 0)</code>	more rotation-accurate gradient kernel
Emboss	<code>[[-2,-1,0],[-1,1,1],[0,1,2]]</code>	3D relief effect

`cv2.filter2D(src, ddepth, kernel)`: `ddepth=-1` means "same depth as the input".

Output size with input N, kernel K, padding P, stride S: $\lfloor (N + 2P - K) / S \rfloor + 1$.

- *Valid* (no padding): $N - K + 1$. *Same*: $P = (K - 1)/2$ so output = input size.
- Example: 5×5 image, 3×3 kernel $\rightarrow$ valid = 3×3 ; with stride 2 $\rightarrow \lfloor (5-3)/2 \rfloor + 1 = 2 \times 2$.

Border handling (`borderType`): `BORDER_CONSTANT` (zeros), `BORDER_REPLICATE`, `BORDER_REFLECT`, `BORDER_REFLECT_101` (**OpenCV default**), `BORDER_WRAP`.

⚠ **Manual code that doesn't do what its heading says (I believe; test it):**

1. `cv2.filter2D` has **no `strides` parameter**: the manual's "valid with strides" call would raise a `TypeError`. Do strides as `result[:, :2, :2]`.
2. `filter2D` always returns the **same size** as the input. A true "valid" output needs cropping: `result[K//2:-(K//2), K//2:-(K//2)]`. The "valid" example (`BORDER_CONSTANT`) is really zero-padding, and the "same (zero padding)" example uses `BORDER_REFLECT`, not zero padding.
3. `cv2.getGaussianKernel(5, 1.0)` returns a **5×1 column vector** (1D), so `filter2D(image, -1, kernel)` blurs only vertically. For a 2D Gaussian: `k = cv2.getGaussianKernel(5, 1.0); k2d = k @ k.T` (or use `cv2.GaussianBlur`).
4. Strictly, `filter2D` computes **correlation** (it doesn't flip the kernel). For symmetric kernels (box, Gaussian) it makes no difference; for Sobel it flips the sign.
5. The manual calls "standard convolution" "full convolution"; in textbook terms *full* means output size $N + K - 1$. If an MCQ quotes the manual, follow the manual; know the real definition.

5.9 Edge detection

An **edge** = rapid change in intensity/colour, usually an object boundary. Edge detection reduces data while preserving structure; it feeds segmentation, detection, and feature extraction.

Method	Idea
Sobel / Prewitt / Scharr	first-derivative gradient operators (3×3); edges where gradient magnitude is high
Canny	multi-stage: smooth, gradient, non-max suppression, hysteresis
Laplacian of Gaussian (LoG) / Marr-Hildreth	Gaussian smooth + second derivative ; edges at zero-crossings
Sobel-Feldman	listed in the manual as emphasising diagonals; I believe it's just another name for the standard Sobel operator, so verify

Method	Idea
CNN-based	learn edge patterns from data

Gradient: G_x = horizontal change (detects *vertical* edges), G_y = vertical change (detects *horizontal* edges). Magnitude $G = \sqrt{G_x^2 + G_y^2}$; direction $\theta = \text{atan2}(G_y, G_x)$. Prewitt uses weights 1,1,1 (Sobel uses 1,2,1, which smooths a bit more).

Canny (John F. Canny, 1986) steps:

1. **Gaussian smoothing:** suppress noise (larger $\sigma \rightarrow$ smoother, loses fine detail).
2. **Gradient calculation:** Sobel $G_x, G_y \rightarrow$ magnitude and direction.
3. **Non-maximum suppression:** keep a pixel only if it's the local maximum along the gradient direction $\rightarrow$ thin (1-pixel) edges.
4. **Hysteresis thresholding:** magnitude $> T_{\text{high}}$ = strong edge (kept); T_{low} to T_{high} = weak (kept *only if connected to a strong edge*); $< T_{\text{low}}$ = discarded.

```
edges = cv2.Canny(blurred, threshold1=100, threshold2=200)    # (image, low,
high, apertureSize=3, L2gradient=False)
```

Rule of thumb: high $\approx 2\text{--}3 \times$ low. **Low thresholds $\rightarrow$ many edges + noise; high thresholds $\rightarrow$ clean but broken edges.**

Simple Sobel edge map (manual): `gradient_magnitude > threshold` then `.astype(np.uint8) * 255`.

Just in case: corner detection. The Lab 4 objectives list "corner detection" but the manual contains no such section (⚠). Quick facts, not from the manual: Harris `cv2.cornerHarris(np.float32(gray), blockSize=2, ksize=3, k=0.04)` with response $R = \det(M) - k \cdot \text{trace}(M)^2$; Shi-Tomasi `cv2.goodFeaturesToTrack(gray, maxCorners, qualityLevel, minDistance)`. A corner = strong gradient change in *two* directions.

5.10 Lab 4 self-test

- HOG vector length for 64×128 , 8×8 cells, 2×2 blocks, 9 bins? (3780.)
- How many bins does uniform LBP with $P = 8$ produce? (10.)
- What's the OpenCV hue range? (0–179.)
- Why `CV_64F` in Sobel? (Keep negative gradients; `CV_8U` clips them to 0.)
- In Canny, what happens to a weak pixel not touching a strong one? (Discarded.)
- First vs second derivative edge criteria? (Peak of gradient vs zero-crossing of Laplacian.)

6. Lab 5: Image segmentation

6.1 What is segmentation?

Image segmentation partitions an image into distinct, non-overlapping regions that each correspond to a meaningful object or part of the scene, grouping pixels by similar colour, texture, or intensity.

The mathematics behind it (manual's four families):

- **Thresholding:** compare intensity to a threshold T : object if $I(x,y) > T$, else background.
- **Edge detection:** find rapid intensity changes = boundaries (Sobel, Canny, LoG).
- **Clustering:** group similar pixels by features (K-Means, Mean-Shift); minimise $\sum \|x_i - \mu_j\|^2$.
- **Graph theory:** pixels = nodes, edges = similarity; minimum spanning tree / graph cuts (min-cut/max-flow) partition the image.

Real-life uses: tumour/organ isolation (medical), pedestrians/cars/signs (autonomous driving), land-cover classification (satellite), cell counting (biomedical), scene understanding (AR).

Method chooser (memorise):

Method	Core assumption	Best when
Global threshold	one intensity cut separates object/background; uniform lighting	clear intensity gap
Adaptive threshold	local neighbourhoods have their own cut	uneven lighting (documents)
Otsu	histogram is bimodal ; picks T automatically	consistent backdrop, unknown T
Colour (HSV) threshold	the object has a distinct colour	intensity alone fails
Canny edges	boundaries = strong intensity jumps	you need outlines, not regions
Region growing	the object is connected and has similar intensity to a seed	known location, homogeneous region
Watershed (markers)	objects are blobs with distance-transform peaks; touching objects	touching/overlapping objects
K-Means	colours form clusters	simplify colour image into K regions

Method	Core assumption	Best when
Deep learning	learned from data	complex tasks, large datasets
Active contours (snakes)	deformable curve settles on gradients	boundary refinement, tracking

6.2 Thresholding family

Global: `_, mask = cv2.threshold(gray, 128, 255, cv2.THRESH_BINARY)` : one T for the whole image.

Adaptive:

```
mask = cv2.adaptiveThreshold(gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                             cv2.THRESH_BINARY, blockSize=11, C=2)
```

- `maxValue` (255): value assigned when the condition holds.
- `adaptiveMethod`: `ADAPTIVE_THRESH_MEAN_C` (threshold = **plain mean** of the neighbourhood – C) or `ADAPTIVE_THRESH_GAUSSIAN_C` (**Gaussian-weighted mean**, centre pixels count more, – C).
- `blockSize`: neighbourhood size, **odd and > 1** (11 → 11×11 window).
- `C`: constant **subtracted** from the local mean: $T(x,y) = \text{local_mean} - C$.
- Input must be an **8-bit single-channel** image. With `THRESH_BINARY`: pixel = `maxValue` if $\text{src} > T(x,y)$ else 0.

Otsu:

```
T, mask = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
print('Otsu picked', T)           # the chosen threshold comes back in the
first return value
```

Otsu tries every possible T and picks the one that **maximises between-class variance** (equivalently minimises within-class variance): $\sigma^2_B = w_0 \cdot w_1 \cdot (\mu_0 - \mu_1)^2$. The `thresh` argument you pass (0) is ignored. Works best when the histogram is **bimodal**; fails with uneven lighting or very unbalanced class sizes (I believe; test on your image).

Colour-based (HSV):

```
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
mask = cv2.inRange(hsv, np.array([30, 50, 50]), np.array([60, 255, 255])) #
lower, upper
sel = cv2.bitwise_and(img, img, mask=mask)
```

`inRange` returns 255 where **all three** channels lie in [lower, upper]. OpenCV hue: red ≈ 0 , yellow ≈ 30 , green ≈ 60 , cyan ≈ 90 , blue ≈ 120 (range 0–179). The manual's example [30,50,50]–[60,255,255] is yellow-green to green. Red wraps around 0, so it needs **two ranges OR'd together**.

Hysteresis (Canny): `cv2.Canny(gray, 100, 200)`: two thresholds, strong + connected weak edges.

6.3 Region growing (manual implementation, understood)

```
def region_growing(image, seed, threshold):
    mask = np.zeros_like(image, dtype=np.uint8)
    stack = [seed]
    seed_intensity = image[seed]
    while stack:
        x, y = stack.pop()
        if x < 0 or x >= image.shape[0] or y < 0 or y >= image.shape[1]:
            continue # out of bounds
        if mask[x, y] == 0: # unvisited
            if abs(int(image[x, y]) - int(seed_intensity)) < threshold:
                mask[x, y] = 255 # accept
                stack.extend([(x+1,y), (x-1,y), (x,y+1), (x,y-1)]) # 4-
    neighbours
    return mask
```

- **Algorithm:** start at the seed → test neighbours → accept those within `threshold` of the **seed's** intensity → expand from accepted pixels only.
- `int(...)` casts prevent `uint8` wrap-around in the subtraction.
- Here $(x, y) = (\text{row}, \text{col})$ because of NumPy indexing; seed (10, 10) = row 10, col 10. If you pick a seed with the mouse ($x = \text{column}, y = \text{row}$), swap them.
- Uses a **stack** (depth-first); a `deque` + `popleft()` gives breadth-first with the same result.
- **Connectivity:** 4 (up/down/left/right) here; 8 would add diagonals.

Why the seed changes the result (Task 6 answer): the acceptance rule is relative to the seed's intensity *and* the region only grows through *connected* accepted pixels. A different seed has a different reference intensity (e.g. grey vs white matter), so a different intensity band is accepted; a seed on a noisy pixel or an edge gives an unrepresentative reference and a tiny or odd region; a seed in a different structure starts a region that may never touch the first one. **Threshold effects:** too low → fragmented region (under-segmentation); too high → **leaks** into neighbouring tissue. Improvements: blur first, or use the mean of a 5×5 neighbourhood as the reference.

6.4 Watershed (marker-controlled), step by step

Idea: treat the image as a topographic surface and flood it from **markers**; where floods from different markers meet you get **boundaries**. Great for touching objects, where a plain threshold gives one merged blob.

```
image = cv2.imread('coins.jpg'); gray = cv2.cvtColor(image,
cv2.COLOR_BGR2GRAY)
# 1-2 Preprocess + threshold. Objects dark on light -> INV; objects light on
dark -> no INV
_, thresh = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV +
cv2.THRESH_OTSU)
# 3 Noise removal: opening = erosion then dilation (kills small white specks)
kernel = np.ones((5, 5), np.uint8)
opening = cv2.morphologyEx(thresh, cv2.MORPH_OPEN, kernel, iterations=2)
# 4 Sure background: dilate -> everything outside is definitely background
sure_bg = cv2.dilate(opening, kernel, iterations=3)
# 5 Distance transform: each white pixel -> distance to nearest black pixel
(centres = peaks)
dist = cv2.distanceTransform(opening, cv2.DIST_L2, 5)
# 6 Sure foreground: keep only pixels far from the edge (coin cores)
_, sure_fg = cv2.threshold(dist, 0.2 * dist.max(), 255, 0)
sure_fg = np.uint8(sure_fg)
# 7 Unknown region = sure background - sure foreground (the uncertain ring)
unknown = cv2.subtract(sure_bg, sure_fg)
# 8 Marker labelling
n, markers = cv2.connectedComponents(sure_fg)
markers = markers + 1 # background becomes 1 (0 is reserved
for 'unknown')
markers[unknown == 255] = 0
# 9 Watershed (image must be 8-bit, 3-channel; markers int32)
markers = cv2.watershed(image, markers)
# 10 Boundaries come back as -1
image[markers == -1] = [255, 0, 0]
print('Coins found:', n - 1) # n counts the background component
too
```

Parameter notes:

- `THRESH_OTSU` finds T automatically. `MORPH_OPEN` with 5×5 kernel, 2 iterations removes noise.
- `distanceTransform(src, distanceType, maskSize)`: `DIST_L2` = Euclidean, `maskSize=5` is the approximation mask.

- The **distance-transform fraction (0.2 in the manual)** controls how much of each object counts as sure foreground. (The OpenCV tutorial I remember uses 0.7, I believe, so the manual's 0.2 is on the lenient side.)
- Why `+1`: `watershed` treats label 0 as "unknown", so the sure background must not be 0.
-  The manual's image is BGR; `[255, 0, 0]` is **blue** in BGR (it only looks red in Matplotlib, which swaps channels, but then the whole picture's colours are swapped). Convert with `cv2.COLOR_BGR2RGB` before `plt.imshow`.

6.5 K-Means clustering segmentation

```

pixels = image.reshape((-1, 3))                # (H*W, 3): each pixel = a
3-D feature vector
pixels = np.float32(pixels)                    # cv2.kmeans requires
float32
criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 100, 0.2)
K = 4
_, labels, centers = cv2.kmeans(pixels, K, None, criteria, 10,
cv2.KMEANS_RANDOM_CENTERS)
centers = np.uint8(centers)                    # back to 8-bit colours
segmented = centers[labels.flatten()].reshape(image.shape)

```

Algorithm: pick K centroids $\rightarrow$ *assign* every pixel to the nearest centroid (Euclidean) $\rightarrow$ *update* centroids = cluster means $\rightarrow$ repeat until assignments stop changing. **Parameters:** `criteria` stops after 100 iterations **or** when centres move < 0.2 (eps); `10` = **attempts** (runs with different initialisations; best is kept); `KMEANS_RANDOM_CENTERS` = random init (`KMEANS_PP_CENTERS` = k-means++). Returns `(compactness, labels, centers)`: labels shape (N,1), centers shape (K,3). **Effect of K:** $K = 2 \rightarrow$ coarse foreground/background-ish split; $K = 4 \rightarrow$ main regions (e.g. sky / foliage / animal / ground); $K = 6 \rightarrow$ more detail, closer to the original. Output has **exactly K colours**. Limits: you must choose K , random init can change results, and it ignores **spatial** position (pixels far apart can share a cluster).

6.6 Deep learning and active contours (definition-level)

- **Deep-learning segmentation:** CNNs for semantic and instance segmentation; frameworks: TensorFlow, PyTorch, Keras. Best for complex tasks and big datasets.
- **Active contour models (snakes):** deformable curves that evolve toward object boundaries using image gradients and smoothness constraints; used for tracking and boundary refinement.

6.7 Lab 5 tasks: solutions and model answers

General requirements: Python + OpenCV + NumPy; show original and result; **no prebuilt segmentation library or deep-learning model**; record and justify parameters; save final outputs (`cv2.imwrite('output/name.png', img)`).

Task 1: uneven-lighting document

```
gray = cv2.imread('data/doc.jpg', cv2.IMREAD_GRAYSCALE)
glob = [cv2.threshold(gray, t, 255, cv2.THRESH_BINARY)[1] for t in (80, 127, 170)] # three T values
adap = cv2.adaptiveThreshold(gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, 31, 10)
# plot Original -> Global T1 -> Global T2 -> Global T3 -> Adaptive (1x5 subplot)
```

Required one-liner: *"Why does a single threshold struggle when illumination changes?"* A single T assumes the background has about the same brightness everywhere. With a shadowed side, the shaded **background** can fall *below* T and be labelled as text (black noise), while on the bright side faded **text** can rise *above* T and vanish (missing characters). Pixel brightness mixes lighting with the surface itself; one global cut cannot separate them. Adaptive thresholding compares each pixel with **its own neighbourhood**, so a gradual lighting change cancels out. Name the failing regions (e.g. "the dark left edge turns into a black blob; the bright right side loses thin strokes") by looking at your own images.

Task 2: adaptive parameter study (original + ≥ 6 results)

```
for m, mname in [(cv2.ADAPTIVE_THRESH_MEAN_C, 'Mean'),
                 (cv2.ADAPTIVE_THRESH_GAUSSIAN_C, 'Gaussian')]:
    for bs, C in [(11, 2), (31, 10), (71, 20)]: # 3 block sizes x 3 C values (paired here to keep 6 panels)
        res = cv2.adaptiveThreshold(gray, 255, m, cv2.THRESH_BINARY, bs, C)
```

Answer pattern (verify on your image):

1. **Neighbourhood too small:** the local mean follows the *stroke itself*, so you see only edges/outlines, hollow or broken characters, and speckle noise.
2. **Too large:** approaches a global threshold: lighting variation returns (shadow blobs), local adaptation is lost, and it's slower. Rule: window should be a bit larger than a character stroke/character.
3. **C increased:** the threshold $T = \text{mean} - C$ drops, so fewer pixels fall below it $\rightarrow$ **cleaner (whiter) background and less black noise**, but thin strokes fade or break if C is too large.

4. **Cleanest combination:** report the one you judged best from your grid (typically a medium block like 31–51 with moderate C).
5. **Mean vs Gaussian:** Gaussian weights the centre more, so it is usually smoother with fewer artefacts, but decide from *your* output.

Task 3: Otsu on a coins-type image

```
T1, m1 = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
hist = cv2.calcHist([gray], [0], None, [256], [0, 256])
# repeat after a change:
adj = cv2.convertScaleAbs(gray, alpha=0.6, beta=0)          # lower contrast
T2, m2 = cv2.threshold(adj, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
print(T1, T2)                                              # report YOUR
measured values
# plot: Original -> Histogram (plt.plot(hist)) -> Otsu mask
```

Why Otsu is useful: it removes manual tuning: the threshold is computed from each image's own histogram by an objective criterion, so it adapts when exposure or object brightness changes between shots. It needs a bimodal histogram.

Task 4: colour sorting (restrictive vs final mask)

```
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
tight = cv2.inRange(hsv, (24, 200, 200), (27, 255, 255))    # too
restrictive (example numbers; tune!)
final = cv2.inRange(hsv, (20, 100, 100), (35, 255, 255))    # captures the
whole yellow car
extracted = cv2.bitwise_and(img, img, mask=final)
```

Why the restrictive mask fails: one object isn't one HSV value. Highlights have low saturation, shadows have low value, and slight hue variation exists across curved paint, so very narrow S/V or H bounds exclude those pixels and the mask has holes and misses parts. Widening S and V lower bounds (while keeping hue narrow, because hue carries the colour identity) captures the full object. HSV is used because it separates **colour (H)** from **brightness (V)**; RGB mixes them.

Task 5: Canny hysteresis study

```
blur = cv2.GaussianBlur(gray, (5, 5), 1.4)
for lo, hi in [(50, 100), (50, 200), (150, 250)]:
    e = cv2.Canny(blur, lo, hi)
```

- **Strong edges:** pixels above T_{high} (clear part outline). **Weak edges:** between thresholds, kept only if connected to strong ones (continuations, fine detail). **Missing edges:** real edges lost because T_{high} was too high (gaps) or weak segments with no

strong neighbour. **Unwanted edges:** texture/noise lines that appear when thresholds are too low.

- Raising T_{high} with T_{low} fixed $\rightarrow$ fewer strong seeds $\rightarrow$ fewer/shorter edges and more gaps. Lowering T_{low} $\rightarrow$ weak edges link more easily but noise sneaks in.

Task 6: region growing on a brain MRI: use `region_growing` from 6.3 with thresholds like 10, 25, 50 and two seeds in different tissues; show masks side-by-side; explain seed dependence as above.

Task 7: touching coins: implement the 10 watershed stages in 6.4 and plot Original $\rightarrow$ Threshold $\rightarrow$ Sure Background $\rightarrow$ Distance Transform $\rightarrow$ Sure Foreground $\rightarrow$ Unknown $\rightarrow$ Final Watershed. Visually check whether touching coins have a boundary line between them.

Task 8: tune the distance-transform threshold

```
rows = []
for i, frac in enumerate([0.2, 0.4, 0.6, 0.8], 1):
    _, fg = cv2.threshold(dist, frac * dist.max(), 255, 0); fg = np.uint8(fg)
    n, mk = cv2.connectedComponents(fg)
    mk = mk + 1; mk[cv2.subtract(sure_bg, fg) == 255] = 0
    mk = cv2.watershed(image.copy(), mk)           # copy: watershed
draws into the image
    rows.append((i, frac, n - 1, len(np.unique(mk)) - 2)) # markers,
separated regions
print(rows)                                           # fill the table from
YOUR numbers
```

Expected qualitative pattern: **low fraction** $\rightarrow$ sure-foreground blobs are big and touching coins stay **merged** (fewer markers); **high fraction** $\rightarrow$ only tiny cores survive, small coins may vanish or an irregular coin may **split** into several markers (over-segmentation). The best range is the one where marker count matches the visually true coin count. Report your own counts; don't reuse mine.

Task 9: K-Means for $K = 2, 4, 6$: run the 6.5 code in a loop; display Original $\rightarrow K=2 \rightarrow K=4 \rightarrow K=6$; for each K record clusters, visual difference, colour simplification (at most K distinct colours vs. up to millions), and which regions the clusters represent (sky, foliage, animal, ground, ...).

Task 10: choose 3 methods for one image: model comparison table

Method	Assumption	Identifies well	Misclassifies	Most influential parameter
Global threshold	uniform lighting, intensity gap	high-contrast objects	shadowed/bright areas	T
Adaptive threshold	local contrast	text under uneven light	flat large areas become noise	blockSize / C
Otsu	bimodal histogram	object vs plain background	textured/uneven backgrounds	(none; but pre-blur)
HSV threshold	distinct colour	coloured objects	shadows/highlights, similar-hue backgrounds	H, S, V ranges
Canny	boundaries = strong gradients	outlines	weak/low-contrast edges, texture	T_low, T_high
Region growing	connected, homogeneous region	one organ/region	leaks past weak boundaries	threshold, seed
Watershed	blob-like objects	touching objects	over-splits irregular shapes	distance-transform fraction
K-Means	colours cluster	colour regions	ignores position, K must be chosen	K

Final deliverable: Original | Method 1 | Method 2 | Method 3 plus a short technical conclusion **based on what you observed** ("X won because the image has property Y, e.g. uneven illumination / touching round objects / strong colour contrast").

6.8 Lab 5 self-test

- What does `C` do in `adaptiveThreshold`? (*Subtracted from the local mean to give the threshold.*)
- Which return value of `cv2.threshold` carries Otsu's T? (*The first.*)
- Why markers + 1 in watershed? (*0 means unknown; background must be non-zero.*)
- What does `cv2.kmeans` need as input dtype? (*float32.*)
- Why does region growing leak? (*Threshold too high; weak boundary between similar intensities.*)

7. Lab 6: Wavelets, boundary detection, Hough transform, SIFT

7.1 Wavelet transforms

Why wavelets? A Fourier transform tells you *which* frequencies exist but not *when/where*. Wavelets give a **time-frequency (or space-frequency)** view by analysing a signal at **different scales**, so they suit **non-stationary** signals (sudden spikes, edges).

Type	Key idea	Families / uses (per manual)
CWT (continuous)	wavelet $\psi(t)$ continuously dilated (scaled) and translated across the signal	Morlet (oscillatory components); seismic, time-frequency analysis
DWT (discrete)	wavelet scaled/shifted in discrete steps; split into approximation + detail	Daubechies, Haar, Symlet; compression, denoising , feature extraction
WPT (wavelet packet)	extension of DWT that also decomposes the <i>detail</i> branches → more sub-bands	complex frequency content, ML features
Inverse	reconstruct the signal/image from coefficients	compression, denoising

DWT in one picture: a signal is split into **approximation** (low-pass, the coarse trend) and **detail** (high-pass, the fine changes). Repeat on the approximation for multiple levels. Tiny Haar example (unnormalised: a = mean, d = half-difference): $[4, 6, 10, 12] \rightarrow$ approx $[5, 11]$, detail $[-1, -1]$; reconstruct $x_1 = a + d$, $x_2 = a - d \rightarrow 4, 6, 10, 12$. For **2D images**, one level gives four sub-bands: **LL** (approximation), **LH**, **HL**, **HH** (horizontal/vertical/diagonal details). Library: PyWavelets (`pywt.dwt2(img, 'haar')` returns `cA, (cH, cV, cD)`). I believe the lab used `pywt`, but the manual shows no code, so confirm in class.

Denoising recipe: decompose → **threshold the detail coefficients** (noise lives mostly in small detail coefficients) → reconstruct.

7.2 Task: anomaly detection in sensor data with wavelets

Expected output (from the task sheet): top plot = sensor data (blue) + denoised signal (orange dashed); bottom plot = **residuals** (red) with **anomalies** as green dots.

```

import numpy as np, pywt, matplotlib.pyplot as plt
np.random.seed(0)
data = np.random.randn(1000) # or load real
sensor data
coeffs = pywt.wavedec(data, 'db4', level=4) # [cA4, cD4,
cD3, cD2, cD1]
sigma = np.median(np.abs(coeffs[-1])) / 0.6745 # noise
estimate from finest detail
thr = sigma * np.sqrt(2 * np.log(len(data))) # 'universal'
threshold
coeffs[1:] = [pywt.threshold(c, thr, mode='soft') for c in coeffs[1:]]
denoised = pywt.waverec(coeffs, 'db4')[:len(data)] # waverec can
be 1 sample longer
residual = data - denoised
anom = np.abs(residual) > 2 * residual.std() # flag points
beyond 2 sigma (my choice; tune)
plt.subplot(2,1,1); plt.plot(data, label='Sensor Data'); plt.plot(denoised,
'--', label='Denoised Signal'); plt.legend()
plt.subplot(2,1,2); plt.plot(residual, 'r', label='Residuals')
plt.plot(np.where(anom)[0], residual[anom], 'go', label='Anomalies');
plt.legend(); plt.show()

```

Logic: the wavelet-denoised signal is the "expected normal behaviour"; whatever it can't explain (the residual) with an unusually large magnitude is an **anomaly**. `mode='soft'` shrinks surviving coefficients toward 0; `'hard'` zeroes the small ones and keeps the rest unchanged. The $\sigma = \text{median}(|d|)/0.6745$ and $\sqrt{2 \ln n}$ threshold are Donoho–Johnstone's standard recipe (I'm confident; verify against slides). The threshold multiplier (2σ vs 3σ) is a design choice.

7.3 Boundary (edge) detection recap, in Lab 6 form

Sobel (first derivative; 3×3 kernels)

- $G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$ answers "is the right brighter than the left?" → detects **vertical** edges. $G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$ answers "is the bottom brighter than the top?" → detects **horizontal** edges.
- **Manual's numeric example:** patch $\begin{bmatrix} 10 & 10 & 50 \\ 10 & 10 & 50 \\ 10 & 10 & 50 \end{bmatrix}$ → $G_x = (-10+50) + 2(-10+50) + (-10+50) = 160$ (large response at a vertical edge); on a uniform patch (all 50) $G_x = 0$.

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
sx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)          # dx=1, dy=0
sy = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)          # dx=0, dy=1
mag = np.uint8(np.clip(np.sqrt(sx**2 + sy**2), 0, 255))    # G = sqrt(Gx^2 + Gy^2)
```

`cv2.Sobel(src, ddepth, dx, dy, ksize)`: **CV_64F** keeps negative values (the sign = direction of intensity change). With **CV_8U**, negatives clip to 0 and half the edges vanish.

Canny (smooth → gradient → non-max suppression → hysteresis):

```
blurred = cv2.GaussianBlur(gray, (5, 5), 1.4)            # sigma = 1.4 is the
manual's choice
edges = cv2.Canny(blurred, 50, 150)                      # lower, upper
threshold
```

LoG (Laplacian of Gaussian): Gaussian blur first (the Laplacian is a *second* derivative and hugely noise-sensitive), then Laplacian; **edges are at the zero-crossings** of the result.

- First derivative: edge = **peak** (Sobel/Canny). Second derivative: edge = **zero-crossing** (LoG).
- Intuition: first derivative = speed; second derivative = acceleration.

```
blurred = cv2.GaussianBlur(gray, (5, 5), 1.4)
lap = cv2.Laplacian(blurred, cv2.CV_64F)                # kernel ~ [[0,1,0],
[1,-4,1],[0,1,0]]
lap_abs = cv2.convertScaleAbs(lap)                       # |lap| as uint8 for
display
```

⚠ The manual's code only displays `|Laplacian|`; it never actually finds zero-crossings. A minimal zero-crossing detector: `sg = np.sign(lap); zc = (sg[:, :-1]*sg[:, 1:] < 0)` (sign change between horizontal neighbours; do the same vertically and combine).
`cv2.convertScaleAbs` = absolute value, scale, convert to uint8 with saturation.

7.4 Hough transform

Idea: turn shape detection into **voting in a parameter space**. Each edge pixel votes for all the shapes (parameter combinations) that could pass through it; real shapes collect many votes → **peaks** in the accumulator; peaks are back-transformed to image-space shapes.

Lines. Manual's parametrisation: $y = m \cdot x + b \rightarrow$ parameter space (m, b) ; each edge pixel votes for every (m, b) that fits; peaks = lines.

- **Practical note (not in the manual):** vertical lines have infinite slope, so OpenCV uses the **polar form** $p = x \cdot \cos\theta + y \cdot \sin\theta$. A point becomes a sinusoid in (θ, p) space; collinear points' sinusoids intersect at one point.
- Always run **edge detection first** (Canny) so only edge pixels vote.

```
edges = cv2.Canny(blurred, 50, 150)
lines = cv2.HoughLines(edges, rho=1, theta=np.pi/180, threshold=150)
# -> [(rho, theta), ...] infinite lines
segs = cv2.HoughLinesP(edges, 1, np.pi/180, threshold=50, minLineLength=40,
maxLineGap=10) # -> [[x1,y1,x2,y2]]
if segs is not None:
    for x1, y1, x2, y2 in segs[:, 0]:
        cv2.line(img, (x1, y1), (x2, y2), (0, 255, 0), 2)
```

- **rho** = distance resolution (px), **theta** = angle resolution (rad), **threshold** = minimum votes to accept a line (higher $\rightarrow$ fewer, stronger lines).
- **HoughLinesP** (probabilistic) returns **segments with endpoints**; extra args: **minLineLength** (shorter segments dropped), **maxLineGap** (max gap to bridge between collinear points). Both return **None** if nothing is found, so check before looping.

Circles. $(x - a)^2 + (y - b)^2 = r^2$; parameter space is **3D (a, b, r)**; votes for centres and radii; peaks = circles.

```
gray = cv2.medianBlur(gray, 5) # blur first:
reduces false circles
circles = cv2.HoughCircles(gray, cv2.HOUGH_GRADIENT, dp=1.2, minDist=30,
param1=100, param2=40, minRadius=10, maxRadius=80)
if circles is not None:
    circles = np.uint16(np.around(circles))
    for a, b, r in circles[0]:
        cv2.circle(img, (a, b), r, (0, 255, 0), 2); cv2.circle(img, (a, b),
2, (0, 0, 255), 3)
    print('Coins:', circles.shape[1]) # shape (1, N, 3)
```

- **dp**: inverse ratio of accumulator resolution (1 = same as image). **minDist**: minimum distance between centres (prevents duplicate circles). **param1**: **upper Canny threshold** used internally. **param2**: **accumulator threshold for centres**: *lower $\rightarrow$ more (false) circles, higher $\rightarrow$ fewer*. **minRadius / maxRadius**: radius search range (narrower = faster/more accurate).
- Returns **(1, N, 3)** array of (x, y, r), or **None**. Hough circles assumes **round** objects; watershed (Lab 5) handles arbitrary touching blobs.

7.5 Lab tasks: lines and circles

Lane detection (Hough lines)

```
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
edges = cv2.Canny(cv2.GaussianBlur(gray, (5, 5), 0), 50, 150)
h, w = edges.shape
roi = np.zeros_like(edges)
cv2.fillPoly(roi, [np.array([(0, h), (w//2 - 50, h//2 + 50), (w//2 + 50,
h//2 + 50), (w, h)])], np.int32)], 255)
edges = cv2.bitwise_and(edges, roi) # keep only the road
area (trapezoid)
segs = cv2.HoughLinesP(edges, 1, np.pi/180, 50, minLineLength=40,
maxLineGap=100)
left, right = [], []
for x1, y1, x2, y2 in (segs[:, 0] if segs is not None else []):
    slope = (y2 - y1) / (x2 - x1 + 1e-6)
    (left if slope < 0 else right).append((x1, y1, x2, y2)) # y points
DOWN: left lane has negative slope
# average/extrapolate each side, cv2.line(overlay, ...), then
cv2.addWeighted(frame, 0.8, overlay, 1, 0)
```

Why the ROI: otherwise Hough picks up buildings, sky, and car edges. Why slope sign splits lanes: in image coordinates (y down), the left lane rises to the right (slope < 0) and the right lane falls (slope > 0).

Coin detection and counting: the `HoughCircles` code above; count = `circles.shape[1]`; try different `param2 / minDist` and report effects.

Screen detection in a computer lab (Hough lines): my suggested approach (the manual gives no code): grayscale → blur → Canny → `HoughLinesP` → keep near-horizontal ($|\text{angle}| < \sim 10^\circ$) and near-vertical ($|\text{angle} - 90^\circ| < \sim 10^\circ$) segments → group them into rectangles (e.g. draw them on a blank mask and use `cv2.findContours` + `cv2.approxPolyDP`, keeping 4-corner shapes of plausible size) → for each rectangle ROI compute mean brightness: **bright = on, dark = off**. **Anomaly** = fewer rectangles in a row than expected (missing screen) or a rectangle in the wrong place.

Smart security system (boundary detection in a zone, real-time): my suggested approach: define the zone polygon; per frame use background subtraction (`cv2.createBackgroundSubtractorMOG2()`) or frame difference, then `Canny / findContours` on the foreground to get object boundaries; test whether a contour's bounding box or centroid is inside the zone with `cv2.pointPolygonTest(zone, (cx, cy), False) >= 0`; if yes → trigger an alarm (print / overlay text / sound) and draw the boundary in red. (Kept deliberately simple; the manual only says "boundary detection".)

7.6 SIFT (Scale-Invariant Feature Transform)

Purpose: detect and describe **local features** that stay recognisable under changes in **scale, rotation** (and partly illumination and viewpoint). Used for object recognition, stitching, tracking.

The 7 stages in the manual (know the order):

#	Stage	What happens
1	Scale-space extrema detection	Build blurred versions $L(x,y,\sigma)$; Difference of Gaussians $D = L(x,y,k\sigma) - L(x,y,\sigma)$ approximates the Laplacian; keypoint candidates are local extrema across space <i>and scale</i>
2	Keypoint localization	A candidate must be the max/min among its 26 neighbours (8 in its own DoG layer + 9 above + 9 below); position refined by a 3D quadratic fit
3	Orientation assignment	Dominant gradient direction in the neighbourhood becomes the keypoint's orientation → rotation invariance
4	Descriptor generation	Gradient-orientation histograms in sub-regions around the keypoint are concatenated → 128-dimensional vector
5	Keypoint matching	Compare descriptors by Euclidean distance
6	Outlier rejection	RANSAC fits a model (affine/perspective) and discards matches that disagree
7	Final matching + homography	The surviving matches estimate the homography for stitching/recognition

Extra detail from Lowe's original paper (not in the manual; I'm fairly confident): low-contrast and edge-like keypoints are also rejected in stage 2; orientation uses a 36-bin histogram; the 128 numbers come from **4×4 sub-regions × 8 orientation bins**.

```
sift = cv2.SIFT_create() # OpenCV >= 4.4
main package, I believe
kp, des = sift.detectAndCompute(gray, None) # (keypoints,
descriptors); mask=None
vis = cv2.drawKeypoints(rgb, kp, None,
flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
print(len(kp), des.shape) # des shape =
(num_keypoints, 128), float32
```

`DRAW_RICH_KEYPOINTS` draws each keypoint as a circle sized by scale with a line for orientation. Each `Keypoint` has `.pt` (x,y), `.size`, `.angle`, `.response`, `.octave`.

Matching + ratio test + RANSAC + bounding box (object recognition):

```
sift = cv2.SIFT_create()
kp1, d1 = sift.detectAndCompute(ref_gray, None)           # reference object
kp2, d2 = sift.detectAndCompute(scene_gray, None)         # test image /
video frame
bf = cv2.BFMatcher(cv2.NORM_L2)                           # L2 = Euclidean
(right for SIFT floats)
matches = bf.knnMatch(d1, d2, k=2)                        # 2 nearest
neighbours per descriptor
good = [m for m, n in matches if m.distance < 0.75 * n.distance] # Lowe's
ratio test
if len(good) >= 10:                                       # need enough
matches (>= 4 minimum for H)
    src = np.float32([kp1[m.queryIdx].pt for m in good]).reshape(-1, 1, 2)
    dst = np.float32([kp2[m.trainIdx].pt for m in good]).reshape(-1, 1, 2)
    H, mask = cv2.findHomography(src, dst, cv2.RANSAC, 5.0) # 5.0 px
reprojection tolerance
    h, w = ref_gray.shape
    corners = np.float32([[0,0],[w,0],[w,h],[0,h]]).reshape(-1, 1, 2)
    box = cv2.perspectiveTransform(corners, H)             # ref corners
projected into the scene
    cv2.polylines(scene_bgr, [np.int32(box)], True, (0, 255, 0), 3)
```

- **Ratio test:** keep a match only if its best distance is clearly smaller than the second-best (here $< 0.75\times$). Ambiguous matches fail it.
- `queryIdx` indexes the first descriptor set, `trainIdx` the second. `findHomography` returns (H, `inlier_mask`).
- **Video object recognition:** wrap this in `while cap.isOpened(): ok, frame = cap.read()`; compute reference keypoints **once** outside the loop.
- **Asset tracking (monitors/keyboards):** build a reference SIFT descriptor set per asset class; for each lab photo match against each reference, and count an asset as present if inliers $\geq$ a minimum; store results in a dict (inventory). Same pipeline, different bookkeeping.

Panorama stitching with SIFT:

```
# kp1,d1 from left image; kp2,d2 from right image; 'good' matches as above
pts2 = np.float32([kp2[m.trainIdx].pt for m in good]).reshape(-1, 1, 2)
pts1 = np.float32([kp1[m.queryIdx].pt for m in good]).reshape(-1, 1, 2)
H, _ = cv2.findHomography(pts2, pts1, cv2.RANSAC, 5.0) # right image
-> left image's frame
pano = cv2.warpPerspective(img2, H, (w1 + w2, max(h1, h2)))
pano[0:h1, 0:w1] = img1
```

Direction check: `findHomography(src, dst)` maps **src** → **dst**, so for "warp image 2 into image 1's space" `src` = points in image 2.

7.7 Lab 6 self-test

- Peak vs zero-crossing: which detector uses which? (*Sobel/Canny: gradient peak; LoG: zero-crossing of second derivative.*)
- What does `param2` do in `HoughCircles`? (*Accumulator vote threshold: lower → more circles.*)
- Why `HoughLinesP` over `HoughLines` for drawing? (*Returns finite segments with endpoints.*)
- How many neighbours in SIFT's extrema test? (*26.*)
- SIFT descriptor size? (*128 floats.*)
- What does RANSAC do in SIFT matching? (*Fits a geometric model, rejects inconsistent matches.*)
- Which stage gives rotation invariance? (*Orientation assignment.*) Scale invariance? (*Scale-space/DoG extrema.*)

8. Exam toolkit

8.1 If you're short on time: priority order

1. **Section 1.2 facts + every ⚠ box** (cheap marks, common traps).
2. **Lab 3 matrices** (what each entry does, DoF, point-pair counts, composition order) and **Lab 2 formulas** (log, gamma, equalization), since these feed MCQs and "write the code" questions.
3. **Function signatures** for: `threshold`, `adaptiveThreshold`, `Canny`, `Sobel`, `warpAffine`, `getRotationMatrix2D`, `getPerspectiveTransform`, `kmeans`, `HoughLinesP`, `HoughCircles`, `SIFT_create`, `BFMatcher`.
4. Write **one full pipeline from memory** per lab (blank cell, no looking).

8.2 Cross-lab trap list

1. `imread` = **BGR**; Matplotlib = **RGB**; missing `cmap='gray'` gives false colours.
2. `imread` returns **None** on failure (no exception).
3. `shape` = (h, w, c); `dsize` / `warpAffine` size = (**w, h**); points = (**x, y**); indexing = [**y, x**].
4. `uint8` arithmetic: NumPy wraps ($250 + 10 = 4$), `cv2.add` saturates (255). Cast to float before log, square, or divide.
5. `cv2.threshold` returns **two** values (`ret, img`); `THRESH_BINARY` uses **strictly greater**; `maxval` need not be 255.
6. `equalizeHist` = single-channel 8-bit only; for colour use Y of YCrCb or V of HSV.
7. Positive angle in `getRotationMatrix2D` = **counter-clockwise**; same canvas → corners cut; to avoid, compute the new bounding box and shift.
8. Linear maps fix the origin; **translation is affine, not linear**; homogeneous coordinates make it a matrix product.
9. Composition `T @ R @ S` applies **S first**; order matters.
10. Affine = 3 point pairs (6 unknowns); homography = 4 pairs (8 unknowns); `findHomography` + RANSAC for more.
11. OpenCV hue range **0–179**; red wraps around 0.
12. `Sobel` with `CV_8U` loses negative gradients: use `CV_64F`, then `convertScaleAbs` / `clip`.
13. `adaptiveThreshold`: `blockSize` **odd** > 1; `T` = local mean – `C`; input must be single-channel 8-bit.
14. Otsu: pass `thresh = 0` and add `cv2.THRESH_OTSU`; the chosen `T` is the **first** return value.
15. Watershed: markers must be **int32**, image **8-bit 3-channel**, background label ≠ 0, boundaries = **-1**.
16. `cv2.kmeans` needs **float32**; labels shape (N,1) → `.flatten()` before indexing centres.
17. `HoughLines` returns (ρ, θ); `HoughLinesP` returns endpoints; both may return **None**.
18. `HoughCircles` `param2` low → more false circles; blur first.
19. SIFT: **128-D** descriptors, **26-neighbour** extrema test, DoG, ratio test, RANSAC; `NORM_L2` for matching.
20. `cv2.imshow` doesn't work inline in Jupyter; videos: `waitKey`, `release()`, `destroyAllWindows()`.

8.3 MCQ practice bank (answers + one-line why)

1. `cv2.imread` returns colour channels in which order? **BGR**.
2. What does `cv2.imread('wrong.jpg')` return? **None**.
3. An image with 480 rows and 640 columns, 3 channels, has `.shape` ? **(480, 640, 3)**.

4. `cv2.resize(img, (200, 100))` gives shape? **(100, 200, 3)**. dsize is (width, height).
5. For `uint8`, `250 + 10` with NumPy vs `cv2.add ? 4 vs 255` (wrap vs saturate).
6. Positive angle in `getRotationMatrix2D` rotates? **Counter-clockwise**.
7. `cv2.threshold(g, 100, 200, THRESH_BINARY)` : what do pixels of value 100 and 101 become? **0 and 200** (strictly greater than T; white = maxval).
8. Which gamma brightens an image? **$\gamma < 1$** .
9. Log transform is mainly used to? **Expand dark intensities / compress bright ones**.
10. Why $s = c \cdot \log(1 + r)$ and not $\log(r)$? **$\log(0)$ is undefined**.
11. A histogram piled up at the far left means the image is? **Dark / underexposed**.
12. `equalizeHist` works on? **8-bit single-channel images**.
13. Histogram equalization maps r to? **$(L-1) \cdot \text{CDF}(r)$** .
14. A linear transformation $p' = Ap$ always maps the origin to? **The origin**.
15. Minimum point pairs for `getAffineTransform` / `getPerspectiveTransform` ? **3 / 4**.
16. Degrees of freedom: rigid, similarity, affine, projective? **3, 4, 6, 8**.
17. Which property does a projective transform *not* preserve? **Parallelism**.
18. Size of the matrix `warpAffine` expects? **2×3** . (`warpPerspective` : 3×3 .)
19. With column vectors, $M = T \cdot R \cdot S$ applies which first? **S (scale)**.
20. Purpose of block normalisation in HOG? **Reduce the effect of lighting variation**.
21. Number of bins for uniform LBP with $P = 8$? **10 ($P + 2$)**.
22. OpenCV hue range for 8-bit images? **0-179**.
23. In Canny, a weak-gradient pixel adjacent to a strong edge is? **Kept**. (Isolated weak pixels are discarded.)
24. Sobel G_x responds most to? **Vertical edges**.
25. Why `cv2.CV_64F` in Sobel? **To keep negative gradient values**.
26. In `adaptiveThreshold`, increasing C (with `THRESH_BINARY`) tends to? **Lower the threshold $\rightarrow$ fewer black pixels / cleaner background (thin strokes may fade)**.
27. Valid `blockSize` ? **Odd integer > 1 (e.g. 11, 31)**.
28. Otsu's method selects T to? **Maximise between-class variance**.
29. Why does Otsu struggle on unevenly lit documents? **Histogram is no longer cleanly bimodal; one global T can't fit all regions**.
30. In marker-based watershed, why add 1 to `connectedComponents` labels? **0 is reserved for unknown; background must be non-zero**.
31. Required dtype for `cv2.kmeans` data? **float32**.
32. LoG marks edges at? **Zero-crossings of the Laplacian of the smoothed image**.
33. Lowering `param2` in `HoughCircles` causes? **More (possibly false) circles**.

34. Why use (ρ, θ) rather than (m, b) in OpenCV's Hough lines? **Vertical lines have infinite slope.**
35. SIFT descriptor dimension? **128.**
36. SIFT extrema test compares a pixel to how many neighbours? **26** ($8 + 9 + 9$).
37. RANSAC's role in matching/stitching? **Robustly fit the transform and reject outlier matches.**
38. Main advantage of wavelets over a Fourier transform? **Localised in both time/space and frequency.**
39. Which Lab-3 transform simulates vanishing points? **Projective (homography).**
40. Difference between `cv2.add` and `cv2.addWeighted`? **add: saturating sum; addWeighted: $\alpha \cdot a + \beta \cdot b + \gamma$ (controllable blend).**

8.4 Likely "write the code" questions: your drill list

Practise each one from a blank cell (target: < 10 minutes each).

1. Load an image safely, convert BGR→RGB, show original + gray side by side. (1.1)
2. Draw a target board with circles and a bounding rectangle; print the canvas centre. (2.3 Task 4)
3. Blur, then crop a centre ROI from both with NumPy slicing; plot them. (2.3 Task 5)
4. Blend two images with `addWeighted`; or draw a semi-transparent caption bar. (2.3 Task 6)
5. Apply log, gamma, and equalization to a dark image and compare histograms. (3.2–3.6)
6. Piecewise-linear contrast stretch with `np.interp` + `cv2.LUT`. (3.4)
7. Rotate by an angle without cropping; or translate/shear with a hand-built matrix. (4.9)
8. Combine scale + rotation + translation into one 3×3 matrix and apply it. (4.9 Tasks 5–6)
9. Perspective-correct a document/sidewalk art from 4 clicked corners. (4.9 Task 8)
10. Compare global vs adaptive vs Otsu thresholding on an unevenly lit page. (6.7)
11. Segment a coloured object using HSV + `inRange` + `bitwise_and`. (6.7 Task 4)
12. Canny with 3 threshold pairs; explain differences. (6.7 Task 5)
13. Region growing from a seed with a threshold (write the function). (6.3)
14. Watershed pipeline to separate touching coins (and count them). (6.4)
15. K-Means colour quantisation for $K = 2, 4, 6$. (6.5)
16. Sobel magnitude + Canny + LoG on one image in a 2×2 subplot. (7.3)
17. Detect lane lines (Canny + ROI + `HoughLinesP`) or count coins (`HoughCircles`). (7.5)
18. SIFT keypoints + ratio-test matching + homography bounding box. (7.6)
19. Wavelet denoise + residual anomaly flags. (7.2)

20. Video loop with `VideoCapture`, per-frame processing, side-by-side display. (3.12)

8.5 Printable cheat sheet (Python + OpenCV only, fits on a page or two)

```

# ----- BASICS -----
img = cv2.imread(p)                                # BGR; None if fail |
cv2.imread(p, 0) = gray
rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)          # also BGR2GRAY, BGR2HSV,
GRAY2BGR, BGR2YCrCb
h, w = img.shape[:2]                                # shape = (h, w, c)
roi = img[y1:y2, x1:x2]                             # crop [rows, cols]
cv2.resize(img, (w2, h2)) / cv2.resize(img, None, fx=.5, fy=.5,
interpolation=cv2.INTER_AREA)
cv2.imwrite('out.png', img)
plt.subplot(r, c, i); plt.imshow(rgb); plt.imshow(gray, cmap='gray');
plt.axis('off'); plt.title('t')
# ----- DRAW -----
cv2.circle(img, (x, y), r, color, -1)               # -1 fill
cv2.rectangle(img, (x1,y1), (x2,y2), color, th)
cv2.line(img, (x1,y1), (x2,y2), color, th)          cv2.putText(img, 't', (x, y),
cv2.FONT_HERSHEY_SIMPLEX, scale, color, th)
cv2.polylines(img, [pts.astype(np.int32)], True, color, th)
cv2.fillPoly(mask, [pts], 255)
# ----- ARITHMETIC / MASKS -----
cv2.add(a, b) cv2.addWeighted(a, al, b, be, gamma) cv2.subtract(a, b)
cv2.bitwise_and(a, a, mask=m) cv2.bitwise_or cv2.bitwise_not
# ----- POINT TRANSFORMS -----
neg = 255 - g                                         log =
c*np.log(1+g.astype(float)), c = 255/np.log(1+g.max())
lut = np.array([(i/255)**gam*255 for i in range(256)], np.uint8); cv2.LUT(g,
lut)          # gamma
lut = np.interp(np.arange(256), [0,r1,r2,255],
[0,s1,s2,255]).astype(np.uint8)                    # piecewise
cv2.normalize(g, None, 0, 255, cv2.NORM_MINMAX)      cv2.equalizeHist(g)
cv2.createCLAHE(2.0, (8,8)).apply(g)
cv2.calcHist([g], [0], None, [256], [0,256])        cv2.applyColorMap(g,
cv2.COLORMAP_JET)
# ----- GEOMETRY -----
M = cv2.getRotationMatrix2D((cx,cy), deg, scale)    # +deg = CCW, 2x3
M = np.float32([[1,0,tx],[0,1,ty]])                # translation | shear
[[1,k,0],[0,1,0]]
cv2.warpAffine(img, M, (w, h))
M = cv2.getAffineTransform(src3, dst3)              # 3 pts, float32
H = cv2.getPerspectiveTransform(src4, dst4); cv2.warpPerspective(img, H, (w,
h))
H, m = cv2.findHomography(src, dst, cv2.RANSAC, 5.0)

```

```

# 3x3: T=[[1,0,tx],[0,1,ty],[0,0,1]]; M = T @ R @ S (S first); pass M[:2] to
warpAffine
# ----- FILTER / EDGES -----
cv2.GaussianBlur(g, (k,k), sigma)  cv2.blur(g, (k,k))  cv2.medianBlur(g, k)
# k odd
cv2.filter2D(g, -1, kernel)          cv2.Sobel(g, cv2.CV_64F, dx, dy, ksize=3)
cv2.Laplacian(g, cv2.CV_64F)
cv2.Canny(g, low, high)              cv2.convertScaleAbs(x)          mag =
np.sqrt(sx**2 + sy**2)
# ----- SEGMENTATION -----
ret, b = cv2.threshold(g, T, 255, cv2.THRESH_BINARY) # _INV, _TRUNC,
_TOZERO; + cv2.THRESH_OTSU (T=0)
cv2.adaptiveThreshold(g, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C|MEAN_C,
cv2.THRESH_BINARY, blockSize_odd, C)
cv2.inRange(hsv, lo, hi)              # H 0-179, S 0-255, V 0-255
cv2.morphologyEx(b, cv2.MORPH_OPEN, np.ones((5,5),np.uint8), iterations=2);
cv2.dilate(b, k, iterations=3)
d = cv2.distanceTransform(b, cv2.DIST_L2, 5); n, mk =
cv2.connectedComponents(fg); mk = mk + 1; mk[unknown==255] = 0
mk = cv2.watershed(img3ch, mk)        # boundaries == -1
_, lab, cen = cv2.kmeans(np.float32(img.reshape(-1,3)), K, None,
(cv2.TERM_CRITERIA_EPS+cv2.TERM_CRITERIA_MAX_ITER,100,.2), 10,
cv2.KMEANS_RANDOM_CENTERS)
seg = np.uint8(cen)[lab.flatten()].reshape(img.shape)
# ----- HOUGH / SIFT -----
cv2.HoughLines(edges, 1, np.pi/180, thr)          cv2.HoughLinesP(edges, 1,
np.pi/180, thr, minLineLength=, maxLineGap=)
cv2.HoughCircles(g, cv2.HOUGH_GRADIENT, dp=1.2, minDist=, param1=100,
param2=40, minRadius=, maxRadius=)
sift = cv2.SIFT_create(); kp, des = sift.detectAndCompute(g, None)          #
des: (N,128)
m = cv2.BFMatcher(cv2.NORM_L2).knnMatch(d1, d2, k=2); good = [a for a,b in m
if a.distance < .75*b.distance]
box = cv2.perspectiveTransform(corners.reshape(-1,1,2), H)
# ----- VIDEO -----
cap = cv2.VideoCapture(path)
while cap.isOpened():
    ok, f = cap.read()
    if not ok: break
    cv2.imshow('w', np.hstack([f, out]))
    if cv2.waitKey(25) & 0xFF == ord('q'): break
cap.release(); cv2.destroyAllWindows()
# ----- FEATURES (skimage) -----
hog(g, pixels_per_cell=(8,8), cells_per_block=(2,2), visualize=True)

```

```

local_binary_pattern(g, P=8*R, R=R, method='uniform')      # P+2 bins
# ----- WAVELETS (pywt) -----
c = pywt.wavedec(x, 'db4', level=4); c[1:] = [pywt.threshold(i, thr, 'soft')
for i in c[1:]]; y = pywt.waverec(c, 'db4')

```

8.6 Exam-day habits (cheap marks)

- **Start each coding answer** with the safe-load check and the BGR→RGB conversion.
- **Label every plot** (`plt.title`) and call `plt.axis('off')` ; examiners read output first.
- **State parameters and why** in one comment (kernel size, thresholds, blockSize, K, seed). Lab 5 explicitly asks for this.
- **Use relative paths**, no absolute paths.
- If an MCQ conflicts with what you know from the ⚠ list, choose the answer that matches the **manual's wording** unless the option is plainly impossible; if it's a code-output question, trace what the code *actually* does.
- Open-book tip: tab your printed sheet by lab; put the Section 1.2 table and function signatures on the front.